

Programación con IA

César San Juan Pastor

I.E.S. Arcipreste de Hita - Dpto. de Informática

Estudio de innovación

Junio - septiembre de 2026

Contenido

Introducción	5
Tema 1 - La IA está aquí	6
Evolución histórica.....	6
Retos para el programador	6
Resumen.....	7
Introducción.....	8
IA generativa	9
IA Generativa vs. a autocompletado de código	12
Flujo de trabajo para el uso de la IA.....	12
Idea inicial y plan de acción.....	12
Generación de código y asistencia	13
Revisión del código y análisis.....	13
Prueba y depuración	13
Documentación y generación de contenido	13
Resumen.....	13
Elección le herramienta de IA generativa adecuada	14
¡Socorro! Que viene la IA.....	14
Tema 2 - Primeros pasos	16
O sí, es de pago	16
Git Hub Copilot	16
Uso del asistente.....	17
Modos del asistente	17
MCP Protocolo de Contexto de Modelos	18
Patrones de uso comunes.....	18
El contexto	18
Hablemos de tokens	18
Ejercicio.....	19
Descripción del problema	19
Entorno de programación	19
Desarrollo.....	20
Tema 3 - Aplicaciones con la IA Generativa	27
Introducción.....	27
Ante todo, orden	27
Elección de un buen prompt	28
Prompt engineering	29
Tipos de prompts	30
Prompts avanzados.....	32
Los anti-prompts	33

Patrones que funcionan en el flujo de trabajo.....	34
Ejemplos de prompts para programadores.....	34
Ejercicio.....	39
Uno	39
Dos	39
Tres.....	40
Práctica	40
Práctica	41
Tema 4 - Generación del esqueleto	42
Introducción.....	42
Conseguir los requerimientos funcionales	42
Creando el esqueleto de la Aplicación	44
Ejercicio.....	47
Uno	47
Dos	48
Tercero	48
Tema 5 - Generación de código base.....	49
Introducción.....	49
Fase 2. Modelo y conexión.	50
Primeros datos.....	50
Preguntas desordenadas únicas	55
Fase 3. Modelo y conexión.	56
Sesión con 35 preguntas.....	56
Un poco más de prompts.	57
Navegando por las 35 preguntas.....	57
Fase 4. Cierre y evaluación.	60
Cierre final, resultados y revisión.....	60
Ejercicio.....	63
Añadir soporte Github	63
Modificar para que se cree un nuevo examen si no hay uno ya existente para cada usuario.....	63
Modificar para mejora el UI	63
Tema 6 - Otras necesidades	64
Depuración con IA	64
Desarrollo de test con la IA.....	64
Fase 5.	65
Tema 7 - Otras consideraciones	68
Comparativa de los diferentes agentes.....	68
Github Copilot.....	70
Una visión al futuro que nos viene	70
Ejercicio.....	71

Anexo I - Recursos.....	72
Bibliografía	72

Introducción

Este documento, apuntes, o como queramos llamarlo se enmarca dentro de un sentimiento nacido en los departamentos de informática relacionados con la educación de toda la Comunidad en el ámbito de la Educación de Formación Profesional.

Está basado en varios libros “medio” actuales (del 2025 en adelante) e información de la red, se puede decir que es mío la maquetación, organización y no más de 15% del texto. Al final del mismo aparecerán todas las referencias usadas.

Está claro que formamos programadores, está claro que la formación no llega a todas las áreas deseadas (menos después de la famosa reforma) y está más claro todavía que la aparición de la IA hace que el alumnado que estamos educando se vaya a enfrentar a retos mucho mayores que hace cinco años.

El alumnado actual no puede quedarse simplemente en programadores junior, tienen que conseguir una mayor cualificación y capacitación que la requerida hasta hace poco. Con este documento me gustaría abordar cómo formar a este nuevo alumnado en el uso de la IA, no sin proponer antes una serie de medidas indispensables:

- Refuerzo en los conocimientos de programación, aumento de las horas de las mismas. Refuerzo de algorítmica y estructura de datos.
- Cambio en el currículo de segundo. Creación de criterios y resultados de aprendizaje relacionados con el uso de la IA en los entornos de programación, a partir de segundo, nunca en primero.
- Desarrollo de proyectos reales en el aula, incluyendo el uso de la IA de forma responsable. Aumento en un trimestre de los estudios, para desarrollar este punto.

Los desafíos a los que se van a enfrentar nuestros alumnos van a determinar la empleabilidad, es indudable que todos hemos notado el efecto de la irrupción de la IA y aventuramos las posibles dificultades. Los siguientes temas abordarán los conceptos bajo mi punto de vista que deberían desarrollar nuestros alumnos para un uso responsable y adecuado en la programación mejorando sus posibles salidas, evitando caer en los populismos actuales de “pídele al Chat GPT que te haga la aplicación, cualquiera puede”.

Ya nos enfrentamos a esta situación en los comienzos de la informática de consumo en la que cualquier persona con un ordenador era un “experto” en el tema, aquellos que vivimos los ochenta y noventa lo recordamos bien. Se demostró que era necesario profesionales cualificados para avanzar de forma correcta.

Espero nos sirva como guía. Toda sugerencia es bienvenida.

César San Juan Pastor

Tema 1 - La IA está aquí

Evolución histórica

En el año 2000, la IA aplicada a la programación comienza con enfoques simbólicos y heurísticos relativamente limitados: sistemas de recomendación de código, generación automática basada en reglas y primeros intentos de machine learning en análisis estático y depuración. En esa etapa, el desarrollo seguía siendo eminentemente manual, aunque herramientas como autocompletado básico y análisis de errores empezaban a incorporar modelos estadísticos sencillos.

A partir de la década de 2010, con el auge del deep learning, la IA dio un salto clave: aparecieron modelos capaces de aprender de grandes repositorios de código (como GitHub) y comprender patrones complejos. Esto permitió la creación de asistentes más avanzados, como completado de código inteligente, detección automática de vulnerabilidades y generación parcial de funciones. Hacia finales de la década, los modelos basados en transformadores (como GPT) empezaron a aplicarse directamente al código, sentando las bases de herramientas como GitHub Copilot, que ya colaboran activamente con el programador.

Desde 2020 en adelante surge el concepto de vibe coding, que describe una forma más intuitiva y conversacional de programar apoyándose en IA generativa: el desarrollador expresa intenciones, ideas de lo que quiere construir y la IA genera, refina y explica el código en tiempo real. Este paradigma transforma la programación en un proceso más creativo y menos técnico, democratizando su acceso y acelerando el desarrollo, aunque también plantea retos en calidad, comprensión del código generado y dependencia de los modelos.

Retos para el programador

La irrupción de la inteligencia artificial generativa de código está transformando profundamente la profesión de programador. Herramientas capaces de generar funciones, aplicaciones completas, pruebas unitarias, documentación y arquitecturas preliminares están automatizando tareas que tradicionalmente requerían horas o días de trabajo humano.

Sin embargo, lejos de eliminar la necesidad de programadores, la IA está redefiniendo el perfil profesional demandado por las organizaciones. Durante los próximos cinco años, los desarrolladores deberán evolucionar desde un rol centrado en la escritura manual de código hacia uno enfocado en diseño, supervisión, validación, integración y gobernanza de sistemas asistidos por IA.

Las plataformas de IA generativa han alcanzado niveles de productividad sin precedentes en tareas como:

- Generación automática de código.
- Refactorización.
- Creación de pruebas.
- Documentación técnica.
- Conversión entre lenguajes.
- Análisis de vulnerabilidades.
- Generación de consultas SQL.
- Diseño preliminar de arquitecturas.

A continuación, elaboramos un informe del posible impacto en los diferentes niveles de experiencia:

<i>Junior</i>	<i>Semi Senior</i>	<i>Senior</i>
Riesgo: Alto	Riesgo: Medio	Riesgo: Bajo
La automatización afecta especialmente a las tareas de entrada y desarrollo rutinario.	La IA incrementará notablemente su productividad y capacidad de entrega.	La experiencia en diseño, arquitectura y toma de decisiones seguirá siendo escasa y altamente valorada.
Estrategia/Oportunidad: Desarrollar rápidamente competencias diferenciadoras.	Oportunidad: Convertirse en supervisor de agentes de desarrollo y sistemas asistidos por IA.	Oportunidad: Liderar equipos híbridos humano-IA y dirigir la estrategia tecnológica.
Competencias clave:	Competencias clave:	Competencias clave:
• Fundamentos sólidos de programación. • Arquitectura de software. • DevOps. • IA aplicada al desarrollo.	• Automatización avanzada. • Supervisión y validación de código generado por IA. • Integración de soluciones IA. • Liderazgo técnico parcial.	• Arquitectura empresarial. • Gobierno de IA. • Estrategia tecnológica. • Gestión de equipos y toma de decisiones complejas.

Resumen

<i>Nivel</i>	<i>Riesgo frente a la IA</i>	<i>Principal adaptación requerida</i>
Junior	Alto	Aprender más allá de la programación básica.
Semi Senior	Medio	Multiplicar productividad mediante IA.
Senior	Bajo	Liderar, diseñar y gobernar sistemas complejos.

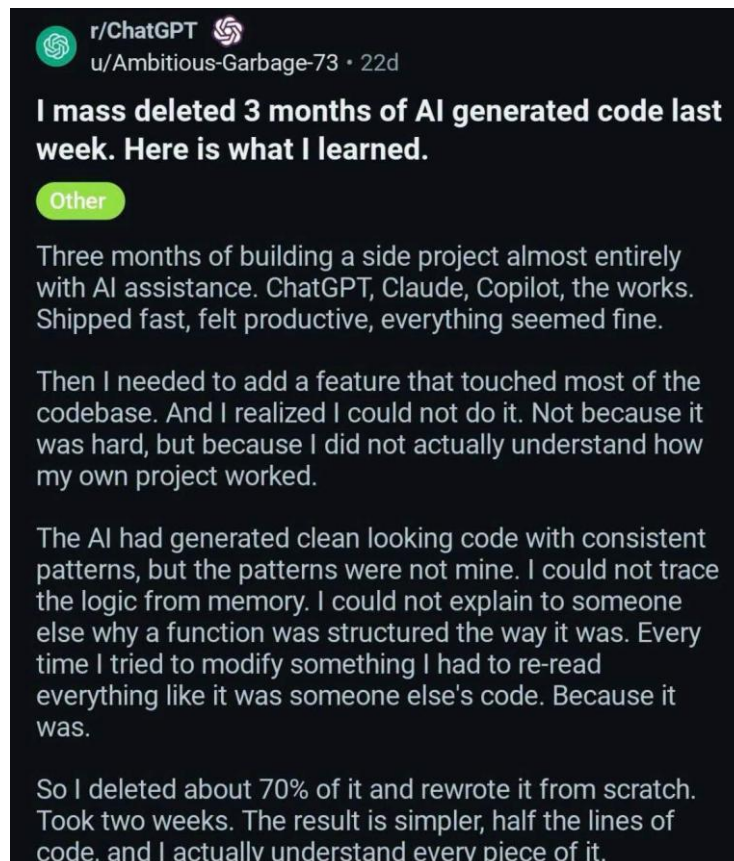
Como consecuencia, las empresas comienzan a buscar menos la capacidad de escribir código, línea a línea y más la capacidad de resolver problemas complejos utilizando herramientas inteligentes.

Para el programador Junior recomendamos encarecidamente que cuando trabaje con IA:

- Comprenda la arquitectura del sistema actual lo suficientemente bien como para especificar los requisitos de forma correcta.
- Revise el código generado para detectar posibles implicaciones de seguridad y casos extremos.
- Garantice que la implementación se mantenga coherente con los patrones existentes.
- Redacte pruebas exhaustivas que verifiquen la lógica de negocio.
- Aprenda los fundamentos: no se salte el “por qué”. Es tentador recurrir a la IA para obtener respuestas a todas las preguntas y no llegar a asimilar nunca realmente los conceptos subyacentes. Utilizar la IA como un tutor, no solo como una máquina expendedora de respuestas. Por ejemplo, cuando la IA te proporcione un fragmento de código, preguntarle por qué ha elegido ese enfoque o pedir que explique el código línea por línea.
- Asegurarse de comprender conceptos como las estructuras de datos, los algoritmos, la gestión de la memoria y la concurrencia sin recurrir siempre a la IA. La razón es sencilla: cuando el resultado de la IA sea erróneo o incompleto, se necesitará nuestro propio modelo mental para reconocerlo y corregirlo
- Practicar la resolución de problemas y la depuración sin la red de seguridad de la IA. Para desarrollar una confianza auténtica, a veces hay que valerse por uno mismo. Muchos

desarrolladores recomiendan celebrar un “día sin IA” o limitar de alguna otra forma la ayuda de la IA de forma periódica. Esto garantiza que sigamos siendo capaces de resolver problemas solo con nuestras propias habilidades, lo cual es importante para evitar la atrofia de las mismas.

- Buscar comentarios y orientación. Una habilidad duradera que acelerará el crecimiento es la capacidad de buscar comentarios y aprender de los demás. No dudar en preguntar a un desarrollador con más experiencia por qué prefiere una solución en lugar de otra.
- Desarrollar las soft skills. Las habilidades de comunicación son tan valiosas como siempre. Practicar a hacer buenas preguntas y describir los problemas con claridad (tanto a los compañeros como a la IA).
- Cambiar la mentalidad: pasar de consumir a crear. Utilizar cada solución que proporcione la IA como un caso de aprendizaje. Analizarla, experimentar con ella y pensar en cómo habrías llegado a ella por ti mismo.



Introducción

La generación automática de código plantea grandes retos en múltiples aspectos de la vida cotidiana. Requiere de un conjunto de habilidades concretas para usarla de forma efectiva, en concreto el saber comunicarse con la IA de forma exitosa, intuir cuándo creer en sus sugerencias, y cuándo integrar o no el código generado en nuestro proyecto.

Las aproximaciones actuales vendidas por múltiples anuncios, de haga Ud. su propia página, nos llevará a generación de código de baja calidad, aumento de los fallos en el código y pérdida de empleo en los primeros estratos de la carrera de programador.

El uso de la IA en la programación puede adoptar varios enfoques, en un extremo del espectro se encuentra el “vibe coding”: es de arriba abajo y exploratoria: se parte de una idea general y se deja que la implementación surja a través de la interacción con la IA. En el otro extremo está lo que se denomina “ingeniería asistida por IA”: un método disciplinado para integrar la IA en cada fase del desarrollo de software, desde el diseño hasta las pruebas, bajo unas restricciones claras. Ambos enfoques aprovechan una IA potente, pero sus objetivos, públicos y expectativas difieren notablemente. No son

categorías mutuamente excluyentes: representan los dos extremos de un espectro, y los flujos de trabajo del mundo real suelen combinar elementos de ambos

Hay que pensar en “vibe coding” como un vehículo exploratorio de alta velocidad: puede llevarnos fuera de los caminos trillados rápidamente, y es ideal para el descubrimiento. La ingeniería asistida por IA se parece más a un tren fiable sobre raíles: primero hay que fijar las vías (planificar), pero es una apuesta más segura y tiene más probabilidades de llegar a un destino definido sin descarrilar. Los desarrolladores de nivel intermedio y avanzado deberían ser capaces de manejar ambos vehículos, pero elegirán en función de la tarea que tengan entre manos. Si el objetivo es innovar o generar ideas rápidamente (por ejemplo, en un hackatón o al validar la viabilidad de una idea), el “vibe coding” aporta impulso. Solo hay que recordar pulir el código si se tiene pensado reutilizar. Si el objetivo es crear una funcionalidad de producto fácil de mantener en un entorno profesional, habrá que inclinarse por la ingeniería asistida por IA, garantiza que no se acabará con código tipo «caja negra» que nadie entiende. realmente.

Yo diría que el futuro ideal es aquel en el que podamos movernos hacia arriba y hacia abajo en este espectro sin esfuerzo alguno.

IA generativa

Es innegable que la irrupción de la IA en la programación está suponiendo y supondrá gran cantidad de retos. En manos de programadores con experiencia puede suponer un aumento de la calidad y cobertura de pruebas entorno al 30%, en manos de programadores sin conocimientos, puede ser un desastre, incorporar código sin supervisión ni comprensión es un riesgo muy elevado.

La IA puede ayudar al programador en diversos campos:

- Puede generar código a partir de sugerencias del programador o documentación técnica.
- Puede analizar el código existente para intentar identificar errores, vulnerabilidades y problemas de rendimiento.
- Puede ayudar con la creación de documentación para los proyectos.
- Puede analizar el código bien cuando se crea, bien al final y sugerir mejoras, buscar elementos redundantes, algoritmos ineficientes y más.
- Puede generar las pruebas unitarias de la aplicación, datos mock y stubs.

La IA generativa se ha incorporado a los IDEs de forma paulatina desde finales de la década anterior. Empezando por las sugerencias de métodos y variables, pasando por la información de tipos en tiempo real, y terminando por la sugerencia de código mientras se programa. Se han diseñado herramientas específicas para los programadores tales como GitHub Copilot, June, etc. que facilitan las labores del desarrollador. Se han creado incluso IDEs completamente nuevos tales como Cursor, Antigravity IDE de Google o Claude Code de Anthropic entre otros muchos, que pueden generar código de forma autónoma leyendo el contexto del nuestro propio.

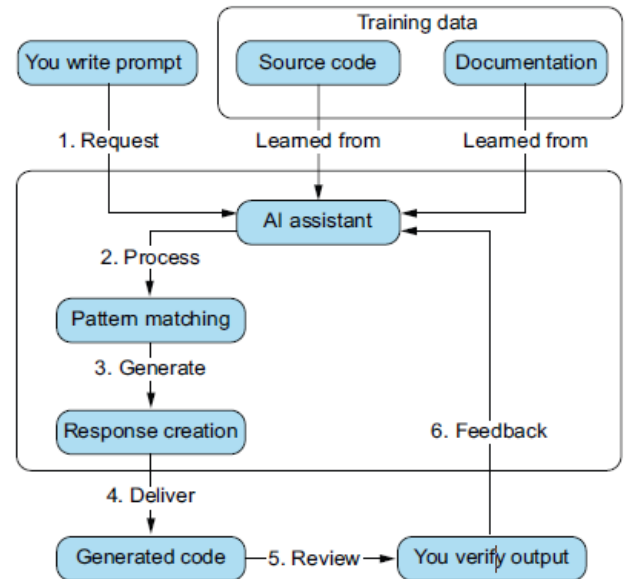
Dentro de las herramientas IA podríamos englobarlas en dos grandes grupos, generalistas y específicas. Dentro de las generalistas encontramos: ChatGPT, Gemini, Copilot. Son agentes que proporcionan acceso a herramientas desarrolladas para el público en general que generan información sobre múltiples campos. Algunas de ellas permiten la integración con otras herramientas de su propio ecosistema, como Copilot con MSOffice. Las específicas serían las desarrolladas concretamente para programación.

La IA generativa está en constante evolución, se nutre de los datos existentes, evidentemente, pero también de los nuevos, y según la vamos utilizando usa el código para adaptarse a nuestra forma de programar, código de estilo, etc. En general la IA está en continuo aprendizaje.

A la derecha observamos un ciclo clásico de generación de código a partir de la IA. Empezando con un modelo entrenado con código fuente, y usando la entrada que facilite el usuario, llegaremos a sugerencias de código, sobre lo solicitado. En los sistemas actuales, agénticos¹, estas sugerencias pueden incluir llamadas al SSOO, tales como creación de directorios, ejecución de programas, e incluso son capaces de desarrollar planes completos de actuación para resolver un problema y llevarlo a cabo, delegando tareas en subagentes.

Pero algo muy a tener en cuenta: **NO son determinísticos**. Es decir, ante la misma entrada no generaran la misma salida.

LLMs



¹ Agéntico, un marco de Inteligencia Artificial capaz de percibir su entorno, razonar y ejecutar acciones de manera autónoma para alcanzar objetivos complejos.

Pero cómo es posible todo este artificio, gracias a los LLM. Un LLM (Large Language Model) es un modelo de inteligencia artificial entrenado con grandes cantidades de texto para entender y generar lenguaje humano. En pocas palabras, aprende patrones del idioma (palabras, frases, contexto) y puede responder preguntas, escribir textos, traducir, programar o resumir información de forma coherente. Suelen funcionar como el “cerebro” detrás de asistentes de IA. Estos modelos usan mecanismos de atención que enfocan al modelo en las partes importantes de las peticiones. Hay que recordar que todo lenguaje de programación es similar en conceptos al lenguaje tradicional o mejor, es mucho más estructurado, con reglas muy definidas.

Algo importantísimo, a tener siempre en cuenta, es que los modelos tienden siempre a generar una respuesta, incluso cuando tienen una precisa, por lo que **pueden producir algo que parece que está muy bien, pero es totalmente falso**. Los LLM no “entienden” el significado de lo pedido, simplemente han aprendido patrones y en base a ellos generan los resultados. Conocen las palabras más frecuentes que van detrás de otras y que código suele ir junto en la programación. **La IA no conoce la respuesta, la está prediciendo**. Es como una gran máquina de pronósticos, que siempre va a sugerir lo que estadísticamente sea más probable basado en los patrones aprendidos, aunque este resultado sea estadístico bajo.

Nota: Sería muy deseable que la IA mostrase información de veracidad estadística cuando genera su información, nos facilitaría muchas decisiones sobre la calidad de lo generado.

Es muy importante destacar la gran diferencia con las BBDD. Estas son deterministas y conocen la respuesta, siempre que se les pregunte lo mismo el resultado será el mismo (si no han cambiado los datos). Pero los LLM no actúan así, son **NO deterministas**, capturan los patrones de los textos a través de múltiples niveles o capas, cada una recogiendo características diferentes, en vez de buscar una respuesta, calcula la probabilidad de cuál será el texto siguiente. Estos modelos, tratan toda la información igual, no hacen juicios de valor, simplemente buscan patrones. Lo que conlleva que puedan aparecer desviaciones, por ejemplo, para programación en Python, tales como:

- Preferencia por patrones subóptimos para resolver el problema.
- Sobreutilización de las librerías incluso cuando existen soluciones más simples.
- Código que falla en los casos límite.
- Uso de patrones desactualizados.

El Contexto

El contexto es crucial para una comunicación efectiva con la IA. El LLM evalúa el contexto buscando relaciones matemáticas entre los elementos. Posteriormente usará esa información para generar la solución.

Pero qué es el contexto. En un LLM, el contexto es toda la información que el modelo tiene en cuenta en un momento dado para generar una respuesta. Incluye el texto previo de la conversación (preguntas, respuestas, instrucciones) y, en algunos casos, datos adicionales proporcionados por el entorno (como documentos, código o historial). Este contexto se introduce como una secuencia de tokens dentro de una “ventana de contexto”, que delimita cuánta información puede procesar junta.

El contexto es crucial porque el LLM no “recuerda” a largo plazo; solo predice sobre lo que está dentro de esa ventana en cada interacción. Cuanto más relevante y bien estructurado sea el contexto, más coherente y precisa será la salida. Por eso, en entornos avanzados como asistentes de programación o vibe coding, se trabaja activamente en seleccionar y organizar el contexto adecuado para guiar al modelo en tareas complejas. Podemos mejorar la generación de código mediante:

- Proporcionando especificaciones detalladas y restricciones.
- Incluyendo el contexto del código (importaciones, dependencias, funciones relacionadas).

- Especificando los requisitos de gestión de errores.
- Declarando las entradas y salidas esperadas.
- Dividiendo las funciones complejas en componentes más pequeños y específicos.

LA HERRAMIENTAS IA SE HAN CONVERTIDO YA EN UNA HERRAMIENTA IMPRESCINDIBLE

IA Generativa vs. a autocompletado de código

Cómo puede un programador hacer uso de la IA Generativa en las tareas que antes realizada el autocompletado. Las principales áreas en la que se está imponiendo la IA en los IDEs:

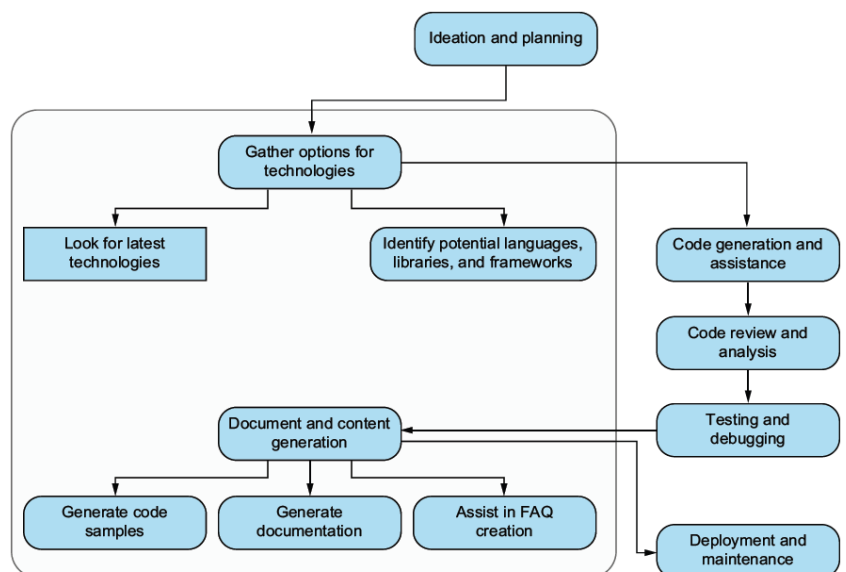
- Autocompletado y sugerencia de código.
- Automatización de las pruebas unitarias.
- Generación de documentación.
- Procesamiento del lenguaje natural.
- Vibe coding. Entendiendo esta forma de programar como un estilo de programación en donde el programador guía a un Agente IA a crear código a través del lenguaje natural en vez de darles especificaciones técnicas complejas. Este tipo de codificación es muy bueno para “aprender”, para “modelar rápido”, para crear prototipos, en donde las ideas se puedan convertir en aplicaciones demostrativas en minutos en vez de horas.

Flujo de trabajo para el uso de la IA

La incorporación de la IA en el flujo de trabajo de la programación permite acelerarla en muchos aspectos, vamos a abordar en cada fase del diagrama anterior cómo ayuda.

Idea inicial y plan de acción

En esta fase del proceso, los agentes basados en LMM y de conversación chat, tales como chat GPT, Gemini, etc. son la mejor opción. Utilizando texto claro, indicando un buen contexto, con un conjunto de listas para las especificaciones tanto Frontend como Backend, e indicando el nivel en el que generar el informe puede dar buenos resultados.



Actuando como arquitecto de software especializado en el diseño y desarrollo de software. Me gustaría crear una aplicación de tareas pendientes en la que pueda añadir una lista de tareas que completar y, después, marcarlas como completadas. ¿Qué pila tecnológica debería utilizar para una versión web de este software?

Podemos usar el estilo conversacional para empezar a esbozar el proyecto, con el fin de planificarlo todo antes de ponernos a programar, explorar ideas, pilas de programación, nuevos conceptos, etc. Es un buen momento para la documentación: como la propuesta del proyecto, las descripciones de las funcionalidades, los requisitos de la aplicación y otros aspectos, lo que permitirá centrarnos más en los objetivos generales.

Generación de código y asistencia

Podemos usar la IA para crear los esqueletos para la aplicación que se esté desarrollando. Permite la generación automática de código, ficheros y directorios para definir toda la base. Aunque pueda automatizar todo el proceso, es imprescindible no utilizar las recomendaciones sin antes entender qué es lo que propone y las consecuencias de implementarlas en nuestro proyecto. Puede que el código que genere sea funcional pero no óptimo, e incluso puede ser dañino.

Crear un esqueleto de API backend para la aplicación en Python

El prompt anterior sería adecuado dentro de la misma conversación en la que hemos generado la idea inicial, ya que aprovecharía ese contexto para darnos el esqueleto de la aplicación de notas.

Revisión del código y análisis

En esta fase podemos usar prompts del estilo:

Evalúa el rendimiento del siguiente código e identifica posibles cuellos de botella: ... el código iría aquí ...

o:

Evalúa el siguiente código buscando problemas de seguridad e identifica posibles vulnerabilidades: ... el código iría aquí...

Prueba y depuración

Podemos usar la IA para generar casos de pruebas unitarias e identificar posibles fallos en el software, no podemos olvidar que la frecuencia con la que los casos extremos son evaluados es bastante menor, por lo que comprobaremos que estos también se incluyen, bien indicándolo en el prompt, bien refinando la generación, junto con nuestra revisión.

Documentación y generación de contenido

La IA es buena en la creación de documentación, definición de resúmenes y cualquier tarea relacionada. Es normal usar las herramientas integradas en el IDE para documentar la aplicación. Se puede usar la salida de la IA como base y refinarlo posteriormente.

Resumen

Uno de los aspectos clave que hay que entender sobre el uso de la IA en la programación es que se trata de un proceso iterativo y colaborativo entre el ser humano y la IA. No basta con escribir una sola indicación perfecta y luego quedarse de brazos cruzados mientras la IA escribe un programa completo sin ningún error. En la práctica, se establece un intercambio constante, un ciclo de retroalimentación que, poco a poco, convierte una idea vaga en un código pulido.

La IA depende del ser humano para recibir instrucciones y validación, y el ser humano delega parte del trabajo a la IA para avanzar más rápido. Es fundamental destacar que la iteración no consiste solo en corregir errores, sino también en hacer evolucionar la solución. Es posible empezar con una indicación

muy general y luego ir perfeccionando progresivamente la intención a medida que vemos lo que produce la IA.

Elección le herramienta de IA generativa adecuada

Antes de hacer una elección y determinar qué herramienta usar en cada fase del proyecto, tendremos que investigar las herramientas existentes, sus puntos fuertes y sus debilidades. Con esta información podremos tomar una decisión sobre el uso. Hay que ser muy conscientes que, en el periodo actual, la evolución de las herramientas es constante, las empresas luchan por llevarse una parte del “pastel” de la IA y cualquier sugerencia quedaría obsoleta a los pocos meses.

Si bien no podemos dar recomendaciones sobre qué herramientas usar, si proponer algunas guías que nos ayuden en la elección:

- Explorar la curva de aprendizaje de cada herramienta.
- Comprobar que el código generado se adapta a los estándares de calidad del proyecto.
- Determinar cómo la herramienta implementa los últimos avances en IA generativa y estar familiarizado con ellos y las nuevas técnicas.
- Seguridad y privacidad con respeto a los requisitos funcionales del proyecto.

¡Socorro! Que viene la IA

Lo primero de todo, no hay que temer a la IA y que el trabajo sea remplazado por ella dejando nuestras habilidades obsoletas. Obviamente, el panorama ha cambiado, sigue cambiando y lo seguirá, haciendo que algunos trabajos desaparezcan, otros cambien y otros nuevos aparezcan. El puesto de programador **NO va a desaparecer de la noche a la mañana**, no vienen hordas de gente sin cualificar a hacer aplicaciones sin intervención del programador. Debemos considerar la IA generativa como un ayudante muy inteligente que puede hacer que la productividad aumente, pero sin nuestra guía no llegará a ningún sitio. Las razones principales que se pueden argumentar son las siguientes:

- La IA no puede resolver problemas, y menos si nunca han aparecido antes. Generar un resultado para producción es una mezcla de conocimiento, intuición y creatividad. La IA no entiende el contexto, lo usa y solo predice.
- Se necesita adaptabilidad. La capacidad humana de adaptarse a los cambios no está implementada en la IA, las habilidades de pensamiento crítico, aprendizaje e innovación es un campo vetado para ella.
- Las consideraciones éticas y morales son muy importantes en nuestra sociedad, no se dejan en manos de algoritmos, por lo menos de momento.
- El toque humano. Aunque esto es poco preciso, y la IA puede generar resultados impresionantes, siguen siendo patrones aprendidos, pero carece de características humanas tales como inteligencia emocional o empatía.

Pero la IA presenta grandes retos. Hay quien dice que estas herramientas pueden fomentar la pereza y dar lugar a que se incorpore código sin revisar a los productos. O que pueden llevar a la gente a escribir software que no entiende. Hay algo de verdad en estos posibles efectos secundarios, y también es posible que genere despidos, pero como toda revolución también genera nuevas oportunidades y está en nuestras manos adaptarnos o morir.

Apreciación personal: Si bien en el momento actual se está generando una pérdida sustancial de puestos de trabajo, estos corresponden principalmente a los puestos iniciales, los que la IA puede sustituir fácilmente (programadores junior). A mi entender esto es similar a generar deuda técnica: ahora las empresas están maximizando resultados, en un futuro, cuando los actuales

puestos superiores no estén, no tendremos fuerza operativa que los pueda cubrir al haber sido sustituidos por la IA, generando una carencia sustancial de puestos medios y superiores.

Nota final: Recientemente han aparecido estudios en los que se sugiere que el uso de la IA conlleva una pérdida de capacidad por parte de la persona. Entendiendo capacidad como inteligencia. La base es que, al hacer uso de estas herramientas el cerebro no se esfuerza lo suficiente, haciéndolo perezoso y cuanto más perezoso es más necesita de la IA, entrando en un bucle sin fin. Estos estudios y sus **supuestas** conclusiones hay que ponerlas en cuarentena a la espera de evidencias firmes. **Como toda herramienta hay que hacer un uso responsable de ella.**

Las personas que no son ingenieros y utilizan la IA para programar se topan con un obstáculo frustrante. Pueden llegar al 70 % del objetivo con sorprendente rapidez, pero ese 30 % final se convierte en un ejercicio de rendimientos decrecientes. ¿Y qué hay de ese último “30”? Esta brecha incluye las partes más difíciles: comprender requisitos complejos, diseñar sistemas fáciles de mantener, gestionar casos extremos y garantizar la corrección del código. En otras palabras, aunque la IA puede generar código, a menudo le cuesta la ingeniería.

Tema 2 - Primeros pasos

0 sí, es de pago

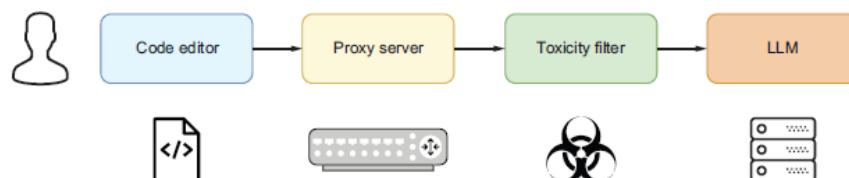
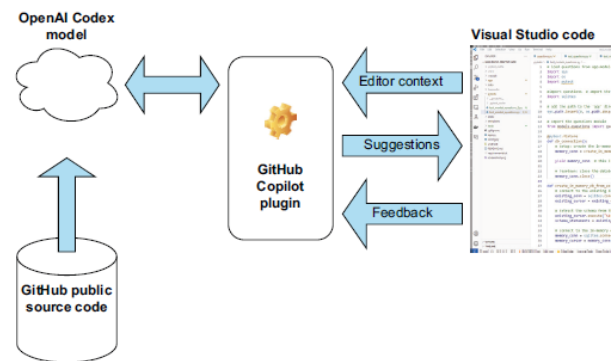
Como todo lo bueno hay que pagar por ello. Los asistentes de programación usan un sistema de tokens para monetizar la infraestructura desplegada y su mantenimiento. Los planes de pago cambian con el tiempo y lo normal es que aumenten con él. Si tienes la suerte de ser docente o alumno puedes acceder al plan Pro de Git Hub Copilot gratuitamente a día de hoy, es necesaria un email corporativo y una cuenta en GitHub. Si no tienes esa suerte, puedes usar el plan gratuito, que todavía existe en estos momentos, pero vaticino que no le quede mucho tiempo.

Git Hub Copilot

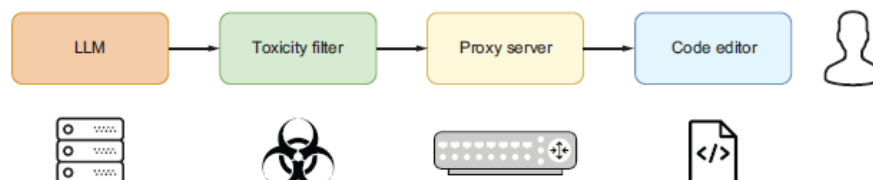
Git Hub Copilot es una herramienta de ayuda, la manera más sencilla de entenderlo es imaginarlo como un ayudante, al que una vez al que le has explicado el problema de forma clara, expondrá posibles soluciones o caminos que ayudarán a encontrar una solución eficiente rápidamente. En concreto, aumenta la productividad y reduce el tiempo en las tareas repetitivas. Es como tener un compañero junior de programación o un pepito grillo que sugiere ideas sobre el trabajo.

La imagen muestra el flujo de trabajo a alto nivel que aparece cuando usamos Copilot desde un IDE. Repasemos el proceso completo.

- **Flujo de entrada:** Conjunto de información que usa el asistente. Junto al contexto se recoge el código anterior y posterior al que se encuentra el cursor, la información sobre las pestañas abiertas, los nombres y tipos de los ficheros abiertos, la estructura del proyecto y las rutas, así como el lenguaje de programación y los Frameworks usados.



- **Flujo de salida:** Es la ruta que lleva la información de vuelta desde el asistente hacia el IDE. Es la principal diferencia que hay con respecto a ChatGPT: el uso del contexto y el filtrado permite generar una salida de mayor calidad.

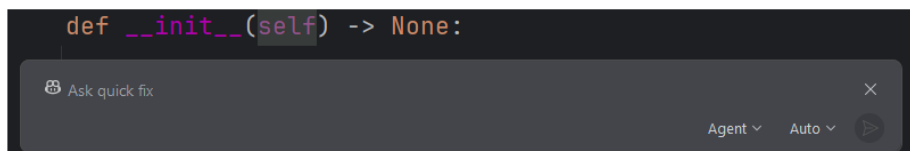
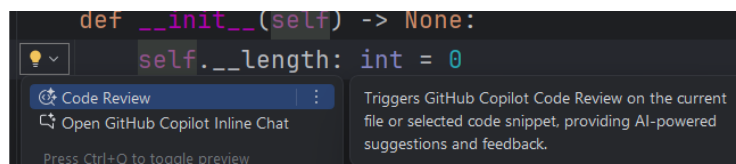
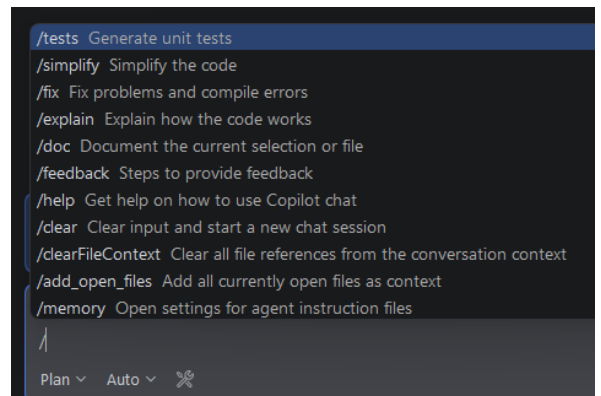
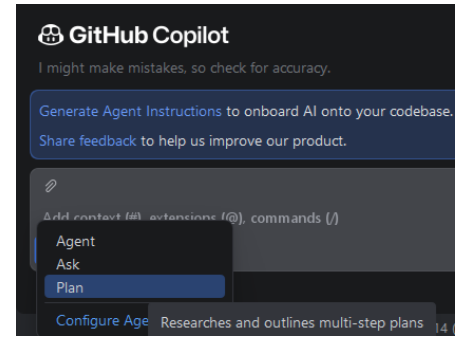


Uso del asistente

GitHub proporciona diferentes formas de uso dentro del editor. Inicialmente se implementaron flujos para poder realizar sugerencias de código y generación de segmentos de código completo. Posteriormente ha evolucionado también a ser un sistema agéntico. Vemos en la imagen los tres modos actuales de funcionamiento: Agente, pregunta o plan.

Pero no solo de código vive GitHub Copilot, lo podemos usar para realizar traducciones de textos, pedirle que busque errores o mejoras de eficiencia en el código e incluso pedirle que genere la documentación por nosotros.

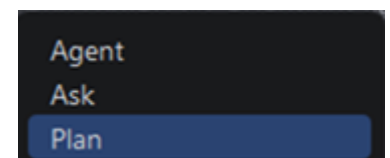
Las acciones que realizaremos con el asistente serán muchas y muy comunes, por lo que nos facilita un conjunto de comandos preestablecidos para indicarle ciertas acciones (prompts). Encontramos todas las opciones disponibles a día de hoy en la imagen de la derecha. Podemos generar test, simplificar código, intentar arreglar los errores de compilación, etc. Estos comandos los usaremos tanto en la ventana de chat de la que hablaremos a continuación, como en el prompt de ayuda que aparece sobre cualquier línea de código al usar el botón derecho del ratón o al pulsar sobre el icono resaltado.



Para finalizar este punto, comentar que donde se despliega la mayor potencia del asistente es a través de la ventana de Chat. Accedemos a ella mediante el icono correspondiente de IDE. En la cabecera de este punto vimos la imagen.

Modos del asistente

El modo “Agent” convierte a GitHub Copilot en un agente autónomo capaz de ejecutar tareas complejas de desarrollo de extremo a extremo. Técnicamente, utiliza modelos de lenguaje junto con herramientas integradas del entorno (edición de archivos, ejecución de comandos, análisis del proyecto, pruebas, etc.) para planificar, modificar código, validar resultados y realizar iteraciones hasta completar un objetivo definido por el usuario.



El modo “Ask” está orientado a la consulta y el análisis. Desde un punto de vista técnico, el modelo recibe contexto del código fuente, archivos abiertos, repositorio o terminal y genera respuestas explicativas, sugerencias o diagnósticos sin realizar cambios directos en el proyecto. Funciona principalmente como una interfaz conversacional enriquecida con conocimiento contextual del entorno de desarrollo.

El modo “Plan” se sitúa entre Ask y Agent. Técnicamente, el modelo analiza el objetivo solicitado, inspecciona el código relevante y genera un plan estructurado de acciones, archivos afectados, dependencias y pasos de implementación antes de ejecutar cambios. Su propósito es ofrecer visibilidad y validación humana del enfoque propuesto, permitiendo revisar o ajustar la estrategia antes de pasar a una fase de ejecución automatizada.

MCP Protocolo de Contexto de Modelos

El MCP ofrece una forma estandarizada para que los modelos de IA detecten e interactúen con herramientas, aplicaciones y fuentes de datos externas. Esto significa que Copilot en el IDE puede conectarse a bases de datos, invocar API, acceder a documentación e integrarse con todo el ecosistema de desarrollo

Patrones de uso comunes

El uso de la IA se ha generalizado, y se ha descubierto que una de las tareas más importantes es cómo facilitar la petición al modelo, es decir, como le damos el prompt. El detalle y la precisión de los datos devueltos dependerán en gran medida de estos datos. Aunque hablaremos más de ello, para abrir boca podemos ir usando los siguientes:

- **Busca el error en el siguiente código.** Resaltando el código y usando la barra integrada.
- **Refactoriza el código para hacerlo un poco mejor.** Refactoriza el código usando los siguientes principios: DRY, ...
- **Genera código para {{{la siguiente tarea}}}**. Crea código para conexiones a BBDD, leer de ficheros, crear peticiones HTTP y muchas más cosas, siendo muy específicos genera buen código.
- **Explica cómo funciona el siguiente código.**
- **Añade pruebas unitarias a este código.**
- **Sugiere mejoras para** facilitar la lectura y el mantenimiento, bázate en las reglas del código limpio.
- **Compara las ventajas y desventajas de la {{{opción 1}}} vs {{{opción 2}}}**

El contexto

El contexto es la información que usa el modelo para realizar sus predicciones. Recoge información de todo el espacio de trabajo, de los ficheros abiertos e incluso de la estructura del mismo. Puede llegar a analizar:

- Código del archivo que se está editando actualmente.
- Contenido de las pestañas abiertas en el IDE.
- Todo el espacio de trabajo o la estructura de la solución.
- Código antes y después de la posición del cursor.

Además, usará toda la información con la que ha sido entrenado, y si el proyecto está alojado en GitHub preguntará por el contexto en el repositorio, los pull-request sobre los problemas abiertos y la documentación y configuración del proyecto.

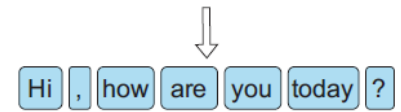
Hablemos de tokens

Los modelos usan información de entrada y generan información de salida. La información se tiene que dividir en pequeños trozos de tamaño similar que el modelo entienda, esos trozos son lo que se denominan tokens. En una asignatura como compiladores e intérpretes, estaríamos definiéndolos como cada uno de los trozos en los que se divide el texto para poder realizar los análisis sintácticos y léxicos. En un modelo, generalmente un token (depende mucho) vienen a ser cuatro o cinco caracteres.

Los tokens tienen dos acepciones, estructural (cada uno de los elementos) y monetaria. Quiero dejar esta distinción bien clara ahora, ya que toda la discusión siguiente solo trata de tokens estructurales.

Los asistentes necesitan entender lo que les pedimos, nosotros les escribimos o les hablamos y esperamos una respuesta, pero no recapacitamos en cómo ellos “entienden” nuestra petición. Ese proceso se hace a través de los modelos NLP, o modelos del lenguaje natural. Su función es recoger una entrada en texto “humano” y hacerla entendible (bits) al modelo. Para eso, el NLP lo primero que hace es tokenizar la entrada, o dividirla en tokens. Los datos se dividen en palabras, signos de puntuación mediante todos los elementos estructurales del lenguaje. A continuación el modelo buscará patrones usando la frecuencia de palabras, los tipos de palabras, la ocurrencias simultáneas de palabras (palabras que aparecen muy frecuentemente juntas) y la clasificación del texto en categorías predefinidas. Con todo este proceso se “hace una idea” de lo que queremos y predice la salida.

Hi, how are you today?



Ejercicio

Vamos a realizar un pequeño proyecto en Python para aclarar los conceptos fundamentales del tema, no vamos a necesitar todavía el Copilot mucho, ya que nuestro fin es entender qué es la tokenización y cómo hacerla nosotros, pasaremos más adelante al uso real.

Descripción del problema

Como primer programa asistido por Copilot, realizaremos un análisis de PLN básico. Tomaremos un archivo de texto del Proyecto Gutenberg y contaremos la frecuencia de las palabras. A continuación, clasificaremos las palabras y veremos cuáles son las más utilizadas. Por último, crearemos y mostraremos un gráfico de barras con los resultados. Todo esto lo haremos con un sencillo script de Python que generaremos nosotros, y utilizaremos GitHub Copilot como nuestro asistente inteligente.

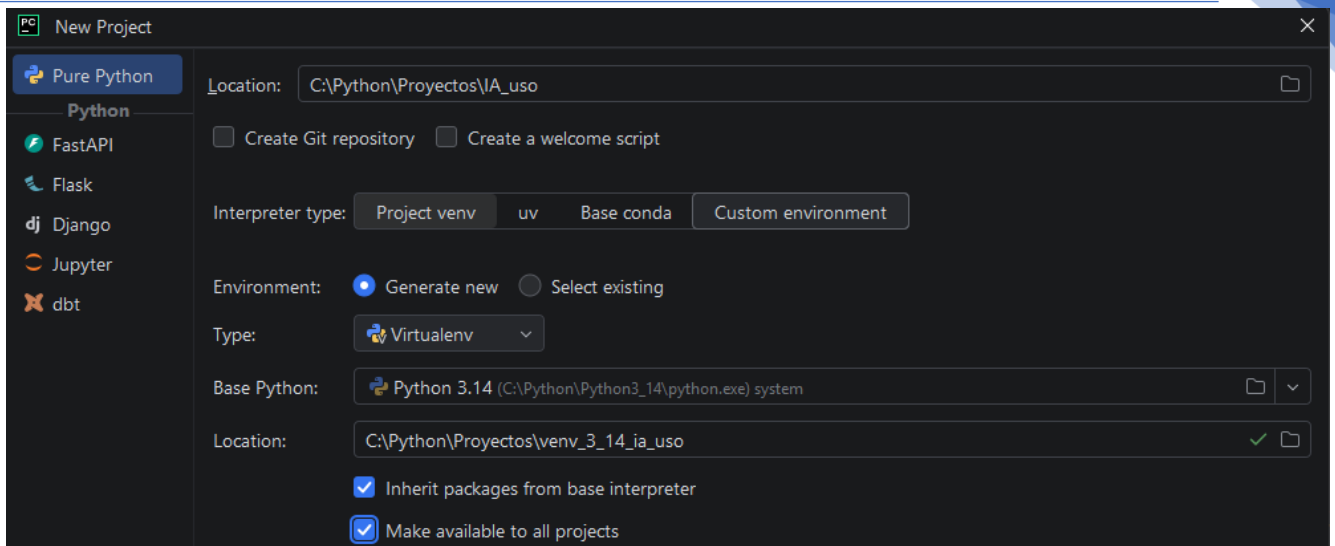
Para el proyecto necesitaremos un texto largo, me he decantado por una de la biblioteca libre Gutenberg.org: <https://www.gutenberg.org/cache/epub/66591/pg66591.txt>

Queremos saber qué palabras se utilizan con más frecuencia en esta novela. Para averiguarlo, contaremos todas las palabras de este archivo de texto y se ordenarán por frecuencia. El pseudocódigo del algoritmo podría ser algo así

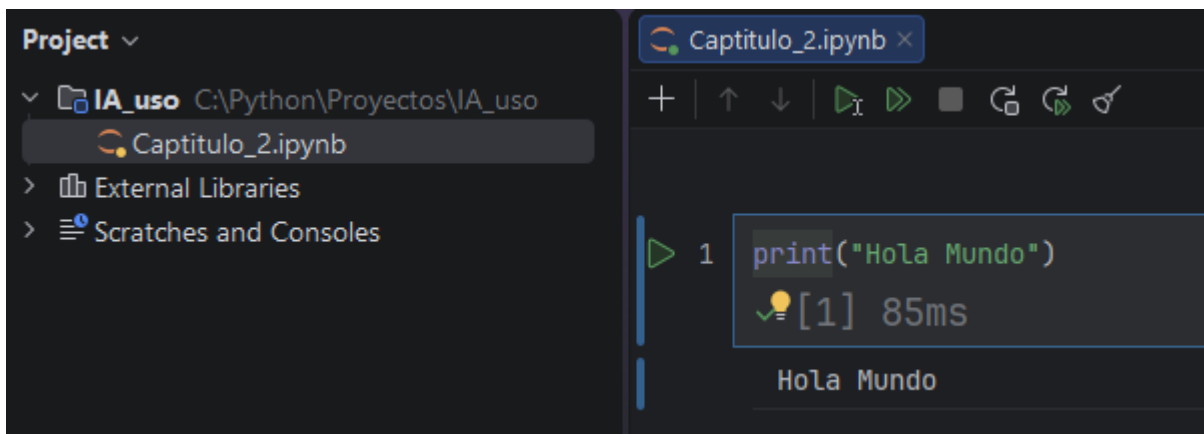
- 1º Leer el fichero
- 2º Eliminar los signos de puntuación y convertirlo a minúsculas
- 3º Tokenizar el texto
- 4º Contar la frecuencia de cada palabra de forma única
- 5º Mostrar las N más frecuentes en un gráfico

Entorno de programación

1. Instalar el IDE deseado, usaremos PyCharm.
2. Instalar Python si no está disponible.
3. Crear un nuevo proyecto Python y un entorno de desarrollo venv asociado.



4. Creación de un Jupyter Note de prueba. Si no arranca es muy posible que haya que instalar todos los paquetes necesarios mediante `pip install`, revisar cualquier instalación básica de Jupyter y de Python.

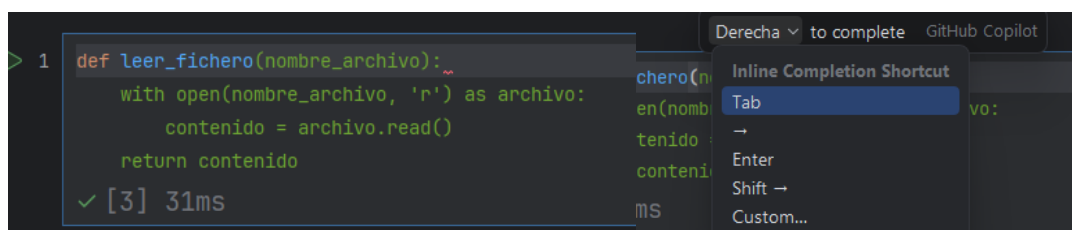


5. Descarga del fichero de texto, usando la url anterior, eliminar de él toda la cabecera de información, empezando solo en el título. En las fuentes de libro se proporciona ya limpio.

Desarrollo

Paso Uno: Lectura del fichero

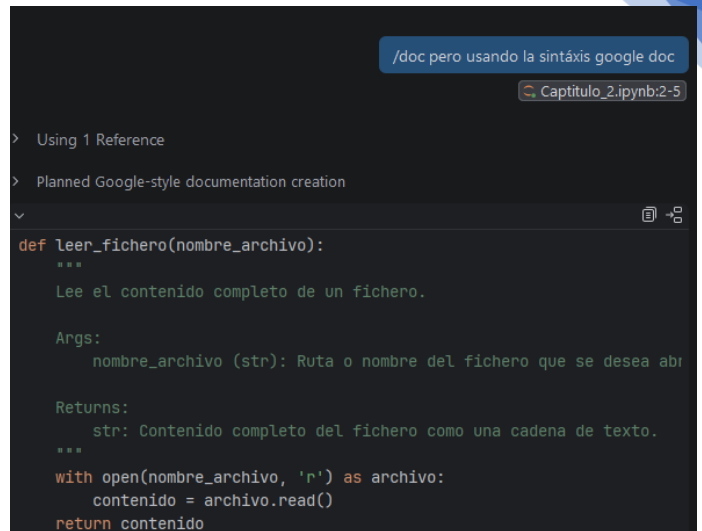
Vamos al paso uno y empezamos a escribir el nombre de la función, nos sugiere todo el código en verde. Al pulsar el cursor (→) se completa el código.



La acción de aceptar la sugerencia es configurable, por defecto es el cursor derecho, pero podemos fijar la que nosotros queramos, ver el desplegable que aparece.

A continuación, documentamos el código como buen programador, ahora **en la ventana chat** le indicamos `/doc` y nos propone una documentación básica, pero nuestro programa usa Google doc, por lo que modificamos el prompt un poco para indicárselo.

Con un resultado aceptable, lo copiaríamos y lo llevaríamos al código. Pero se lo podemos pedir también al Cópilot, si no hemos tocado nada sigue en modo Agente, puede interactuar directamente con el IDE, lo hacemos (Pon el resultado en el código).



```

/doc pero usando la sintaxis google doc
Captitulo_2.ipynb:2-5

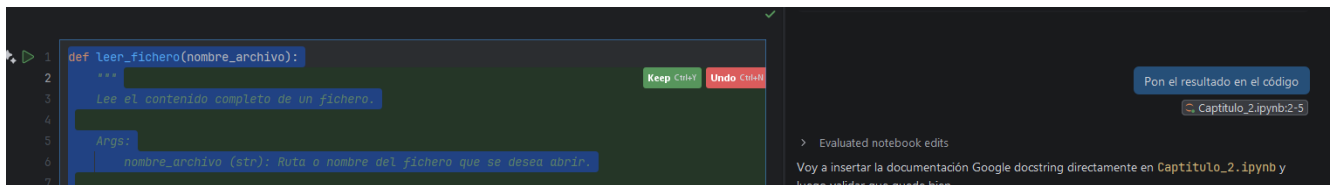
> Using 1 Reference
> Planned Google-style documentation creation

def leer_fichero(nombre_archivo):
    """
    Lee el contenido completo de un fichero.

    Args:
        nombre_archivo (str): Ruta o nombre del fichero que se desea abr

    Returns:
        str: Contenido completo del fichero como una cadena de texto.
    """
    with open(nombre_archivo, 'r') as archivo:
        contenido = archivo.read()
    return contenido

```



```

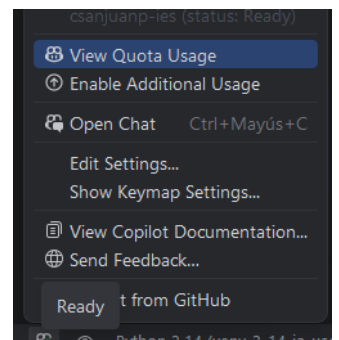
def leer_fichero(nombre_archivo):
    """
    Lee el contenido completo de un fichero.

    Args:
        nombre_archivo (str): Ruta a nombre del fichero que se desea abrir.

```

Aparecen en verde los cambios que se van a realizar, si eliminase alguna línea esta estaría en rojo. Como es muy obediente, no hace nada sin nuestro permiso, está esperando la confirmación con el botón verde **keep**. Las confirmaciones se pueden hacer todas a la vez si aceptamos en la ventana de Chat, o una a una en su correspondiente botón.

Si accedemos al icono inferior de GitHub Copilot y pulsamos sobre la “Quota” usada, vemos que con solo estas acciones hemos gastado ya dos créditos de los 1500 que tenemos



Paso dos: Limpieza

Seguimos en la misma conversación en el chat:

Crea una función que convierta el texto a minúsculas y elimine todos los signos de puntuación usados por el idioma español

Tras unos instantes nos muestra la función y le indicamos

Pon el resultado en el código

El asistente añade el código en su sitio y nos pregunta si lo queremos mantener (keep). Después, sin escribir nada, simplemente añadiendo una nueva celda ya sugiere:

```
✓ [4] 19ms

def contar_palabras(texto):
    """
    Cuenta la frecuencia de cada palabra en un texto.

    Args:
        texto (str): Texto de entrada.

    Returns:
        dict: Un diccionario donde las claves son palabras y los valores son sus frecuencias.
    """
    palabras = texto.split()
    frecuencia = {}
    for palabra in palabras:
        if palabra in frecuencia:
            frecuencia[palabra] += 1
        else:
            frecuencia[palabra] = 1
    return frecuencia
```

Pero volvamos a la función creada

```
def normalizar_texto(texto):
    signos = string.punctuation + "¡¿"
    tabla = str.maketrans("", signos)
    return texto.lower().translate(tabla)
```

Utiliza llamadas a **maketrans** en vez de **replace**, nos parece algo extraño y vamos a indagar un poco en la razón de fondo, cambiando el modo a Ask le pedimos:

explica a nivel técnico la diferencia de usar **str.maketrans** frente a **str.replace**, en cuanto a velocidad y eficiencia

Nos da una explicación completa y nos sugiere que nos puede hacer test de rendimiento, a lo que asentimos

Haz la microcomparación con timeit

Tras copiar el código generado en otra celda y ejecutarlo accedemos a los resultados: en cambios simples no hay diferencia importante, mientras que en cambios múltiples el factor es diez veces menor, por lo que decidimos dejarlo.

Paso tres: Tokenización

Como intuimos que este paso es muy fácil, simplemente añadimos una nueva celda y empezamos a definir nuestra función con *tokeniza_palabras*, nos sugiere

```
=== Reemplazo simple ===
replace: 0.044966 s
translate: 0.061101 s

=== Múltiples reemplazos ===
replace: 0.232300 s
translate: 0.061590 s
```

el contenido. También escribimos unos test para ver cómo va funcionando: una vez definido el texto de prueba, lo normalizamos, lo imprimimos, lo tokenizamos y lo volvemos a imprimir.

```
def tokeniza_palabras(texto):
    return texto.split()
```

```
texto = "Python es un lenguaje de programación muy popular."
texto = normalizar_texto(texto)
print(texto)
texto = tokeniza_palabras(texto)
print(texto)
```

El Copilot nos sigue ayudando durante el desarrollo incluso sin invocar ninguna ventana o Chat directamente. Ya llevamos cinco créditos gastados.

Paso Cuatro: Recuento

Empezamos a escribir el nombre de la función y con solo unos caracteres ya nos muestra un posible contenido. Usa un diccionario y cuenta correctamente, podemos intentar algunos test, sigamos escribiéndolos a continuación.

```
texto = "Python es un lenguaje de programación muy popular. Python es fácil de aprender."
texto = normalizar_texto(texto)
palabras = tokeniza_palabras(texto)
frecuencia = contar_frecuencia_palabras(palabras)
print(frecuencia)
✓ [9] 28ms
{'python': 2, 'es': 2, 'un': 1, 'lenguaje': 1, 'de': 2, 'programación': 1, 'muy': 1, 'popular': 1, 'fácil': 1, 'aprender': 1}
```

```
def contar_frecuencia_palabras(texto):
    frecuencia = {}
    for palabra in texto:
        if palabra in frecuencia:
            frecuencia[palabra] += 1
        else:
            frecuencia[palabra] = 1
    return frecuencia
```

Paso Cinco: Visualización

Ahora le indicaremos que cree una función para mostrar en un gráfico las N palabras más frecuentes.

Crea una función a la que le pasemos un diccionario de palabras con sus frecuencias, y un número de palabras, que las muestre un gráfico de barras con la librería Matplotlib

Este prompt es mucho más interesante y muestra el modo Agéntico en profundidad.

```

1 import matplotlib.pyplot as plt
2
3 def graficar_frecuencias(frecuencias, n_palabras):
4     """
5     Muestra un gráfico de barras con las N palabras más frecuentes.
6
7     Args:
8         frecuencias (dict): Diccionario con palabras como clave y su frecuencia como valor.
9         n_palabras (int): Número de palabras más frecuentes a mostrar en el gráfico.
10    """
11    # Ordenar por frecuencia descendente y tomar las N primeras
12    palabras_ordenadas = sorted(frecuencias.items(), key=lambda x: x[1], reverse=True)[:n_palabras]
13    palabras, conteos = zip(*palabras_ordenadas)
14
15    fig, ax = plt.subplots(figsize=(max(8, n_palabras), 5))
16
17    barras = ax.bar(palabras, conteos, color="steelblue", edgecolor="white")
18
19    # Etiqueta de valor encima de cada barra
20    for barra, conteo in zip(barras, conteos):

```

Crea una función a la que le pasemos un diccionario de palabras con sus frecuencias, y un número de palabras, que las muestre un gráfico de barras con la librería Matplotlib

> Planned function for displaying bar chart with Matplotlib
 Añado la función directamente al notebook existente.
 ✓ Read Capitulo_2.ipynb successfully.
 ✓ Edited Capitulo_2.ipynb +42 -0 successfully.
 ✓ Ran get_errors tool
 Matplotlib no está instalado. Lo instalo ahora:

Run Command

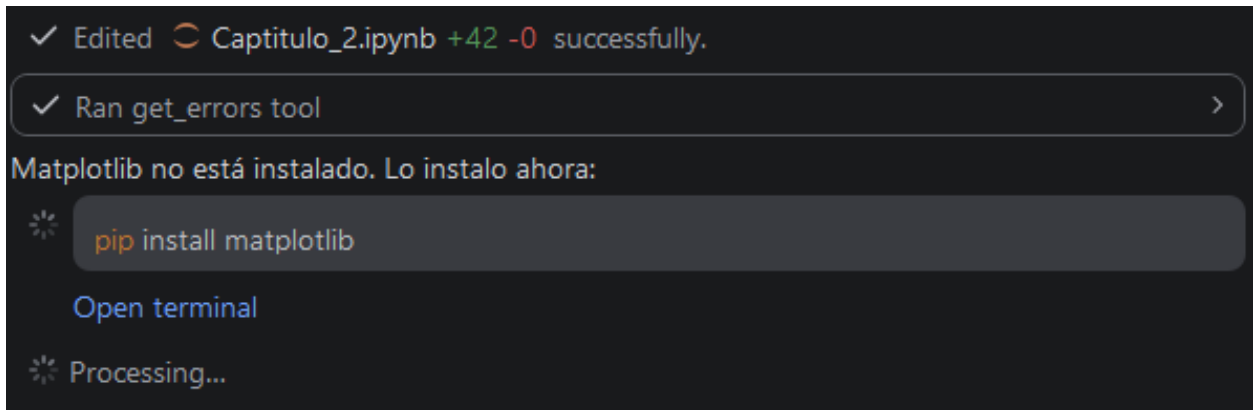
pip install matplotlib

Allow Once
Skip

⚙️ Waiting for your response

No solo nos da un resultado muy aceptable, además detecta que la librería Matplotlib no está instalada y nos pide como primer paso, permiso para instalarla, en la venta del Chat con **Allow Once**, le

indicaremos si puede interactuar con el IDE y el Shell una vez, no puede o a partir de ahora que no pida permiso, esto último no es recomendable bajo ninguna circunstancia. Ejecuta la orden y lo instala.



Tras aceptar los resultados y ejecutarlos aparece un gráfico bastante aceptable, habría que retocar aspectos visuales, pero para este ejemplo lo dejaremos así. Solo una aclaración más, Copilot ha creado los datos de prueba a partir de los que hicimos en la celda anterior, podemos comprobar como el contexto no solo abarca al código actual, también mira alrededor del mismo.

Todo junto

Le pedimos que termine el trabajo, lea el fichero, normalice, tokenice y muestre las 10 primeras palabras. Desde el modo agente.

Crea código usando las funciones de este fichero, para leer `#file:pg66591.txt`, limpiarlo, tokenizarlo, crear la tabla de frecuencias y mostrar las 10 palabras más frecuentes

El código que propone:

```
# — Pipeline completo sobre pg66591.txt

# 1. Leer el fichero
contenido = leer_fichero("pg66591.txt")

# 2. Normalizar (minúsculas + eliminar puntuación)
contenido_limpio = normalizar_texto(contenido)

# 3. Tokenizar
tokens = tokeniza_palabras(contenido_limpio)

# 4. Calcular frecuencias
frecuencias_libro = contar_frecuencia_palabras(tokens)

print(f"Total de palabras      : {len(tokens)}")
print(f"Vocabulario único (tokens) : {len(frecuencias_libro)}")

# 5. Mostrar las 10 palabras más frecuentes en un gráfico de barras
graficar_frecuencias(frecuencias_libro, n_palabras=10)
```

Ejecutamos la celda. Ups, algo ha fallado.

```

17
18 # 5. Mostrar las 10 palabras más frecuentes en un gráfico de barras
19 graficar_frecuencias(frecuencias_libro, n_palabras=10)
X [17] 41ms

-----
UnicodeDecodeError                                Traceback (most recent call last)
Cell In[17], line 4
      1 # --- Pipeline completo sobre pg66591.txt -----
      2
      3 # 1. Leer el fichero
----> 4 contenido = leer_fichero("pg66591.txt")
      5
      6 # 2. Normalizar (minúsculas + eliminar puntuación)
      7 contenido_limpio = normalizar_texto(contenido)

```

Pues parece que algo ha fallado, vamos a preguntarle

```

> Using 1 Reference
v Thinking
describe more precisely what they want to be fixed."
Así que debo ser honesto: no detecto errores de compilación/sintaxis en el código. Pero
puedo ver problemas potenciales:

1. Encoding issue: El fichero pg66591.txt probablemente esté en inglés (es un
proyecto Gutenberg), pero normalizar_texto() añade sign

```

Aporta alguna sugerencia, pero no encuentra la solución, vamos a intentar un prompt más específico.

En la función de lectura del fichero apare el ---- aquí va el código copiado del error ---- solúcionalo

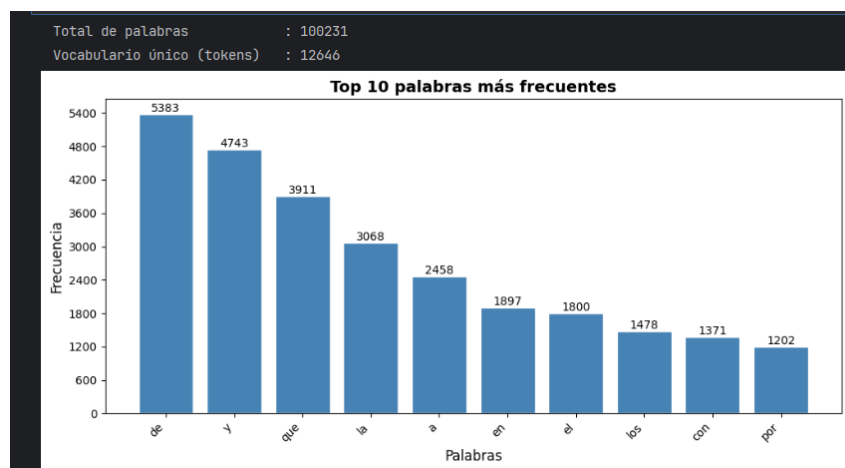
Propone un cambio que vamos a aceptar y probar (anticipamos que es muy probable que sea el error, la experiencia así lo marca).

```

9 str: Contenido completo del fichero como una cadena de texto.
10 """
11 with open(nombre_archivo, 'r') as archivo:
12 with open(nombre_archivo, 'r', encoding='utf-8') as archivo:

```

Ahora sí.



Con un poco de esfuerzo y nuestra guía hemos llegado a la solución, hemos gastado 30 de 1500 créditos disponibles, para un trabajo sencillo el plan de pagos es aceptable, en el momento que necesitemos más contexto y proyectos más grandes, necesitaremos actualizar el plan y €€€€.

Apreciación personal: Hemos llegado a la aplicación final, pero ni mucho menos ha sido un proceso automático y completo, lo hemos guiado y utilizado a nuestro antojo, no hemos permitido que se hagan cambios sin autorización y en todo momento hemos estado presentes. Lo que quiero decir, como se ve, es que la IA ayuda y mejora, pero solo va a transformar el tipo de trabajo a realizar, focalizándonos más en el problema y menos en picar código.

Bueno, al final hemos utilizado la IA un poco más de lo esperado, qué se le va a hacer.

Tema 3 - Aplicaciones con la IA Generativa

Introducción

Se ha observado dos patrones distintos en la forma en que los equipos están aprovechando la IA para el desarrollo. Llamémoslos los “bootstrappers” y los “iteradores”. Ambos están ayudando a los ingenieros (e incluso a los usuarios sin conocimientos técnicos) a reducir la distancia entre la idea y la ejecución (o MVP). En primer lugar, están los “bootstrappers”, que suelen llevar un nuevo proyecto desde cero hasta el MVP. Herramientas como Bolt, v0 y la IA de “captura de pantalla a código” están revolucionando la forma en que estos equipos ponen en marcha nuevos proyectos. Estos equipos suelen:

- Empezar con un diseño o un concepto aproximado.
- Utilizar la IA para generar un código base inicial completo.
- Obtener un prototipo funcional en cuestión de horas o días, en lugar de semanas.
- Se centran en la validación y la iteración rápidas.

Los «iteradores» utilizan herramientas como Cursor, Cline, Copilot y Windsurf para su flujo de trabajo diario de desarrollo. Esto es menos llamativo, pero potencialmente más transformador. Estos desarrolladores:

- Utilizan la IA para la finalización de código y las sugerencias.
- Aprovechan la IA para tareas complejas de refactorización.
- Generan pruebas y documentación.
- Utilizan la IA como «programador de pareja» para la resolución de problemas.

Pero aquí está el quid de la cuestión: aunque ambos enfoques pueden acelerar drásticamente el desarrollo, conllevan costes ocultos que no son evidentes a primera vista.

Cuando vemos a un ingeniero sénior trabajar con herramientas de IA como Cursor o Copilot, parece magia. Son capaces de crear estructuras completas de funcionalidades en cuestión de minutos, con pruebas y documentación incluidas. Pero si observamos con atención, nos daremos cuenta de algo fundamental: no se limitan a aceptar lo que sugiere la IA. Están refactorizando constantemente el código generado en módulos más pequeños y específicos. Añaden un manejo exhaustivo de errores y de casos extremos que la IA ha pasado por alto, refuerzan sus definiciones de tipos e interfaces, y cuestionan sus decisiones arquitectónicas. En otras palabras, están aplicando años de sabiduría en ingeniería, ganada a pulso, para dar forma y limitar el resultado de la IA. La IA acelera su implementación, pero es su experiencia lo que hace que el código sea mantenible.

Ante todo, orden

El diseño es fundamental en el desarrollo de software, pero los programadores suelen pasarlo por alto. Dedicar tiempo a planificar la arquitectura de una aplicación se traduce en usuarios satisfechos y menores costes de mantenimiento. ¿Cómo puede la IA generativa acelerar el proceso de diseño?

Para adentrarnos en cómo usar la IA para el diseño, usaremos el siguiente supuesto: diseñar una aplicación web que resuelva un problema habitual Prepararse para un examen de certificación. La aplicación ofrecerá pruebas de práctica aleatorias para la preparación del examen. En primer lugar, analizaremos el problema y crearemos un documento de diseño con la ayuda de ChatGPT. A continuación, refinaremos el documento y elaboraremos historias de usuario que sirvan de guía para el desarrollo. Repetiremos el proceso para GitHub Copilot con el modo agéntico.

Antes de nada, definiremos un poco el dominio del problema. En concreto queremos hacer:

- Prepararnos para el examen realizando pruebas de prácticas.
- Resolver preguntas similares a las del examen oficial.
- Responder a esas preguntas de forma aleatoria en sesiones de 35 preguntas.

Para ello deberíamos:

- Crear una aplicación web.
- Disponer de un banco de preguntas de examen con sus respuestas.
- Almacenar las preguntas y respuestas en algún lugar.
- Disponer de una interfaz para visualizarlas.
- Seleccionar 35 de ellas al azar para un examen.
- Controlar el curso del examen.
- Calificar el examen.

Ya comentamos en el tema anterior, que para que la IA genere buenos resultados, el prompt elegido es muy importante. Por ejemplo, para empezar a comprender el problema, podríamos usar algo del estilo:

- ¿Cuáles son los casos extremos más habituales al desarrollar {tipo de aplicación}?
- ¿Qué aspectos normativos hay que tener en cuenta en el ámbito de {ámbito}?
- ¿Qué requisitos no funcionales suelen ser importantes para {tipo de aplicación}?

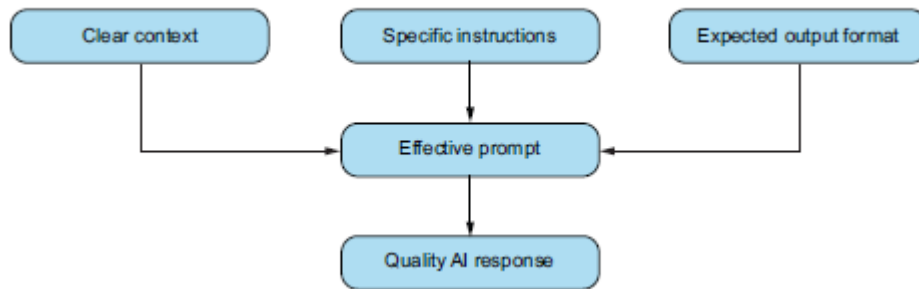
Adentrémonos en este último punto, un buen prompt.

Elección de un buen prompt

La creación de un prompt, ya lo hemos repetido varias veces, es una tarea a la que hay que dedicar tiempo. Además, debería ser un prompt más o menos estructurado para generar buenos resultados, se ha comprobado que este tipo de prompt son mejores que si tratamos a la IA con un estilo conversacional más tradicional. Así un buen prompt empezará fijando el “rol” con el que la IA debe trabajar y a continuación añadir la información que nos parezca relevante. Todo buen prompt debería incluir la siguiente información, incluso estructurada de esta manera, pero no es necesario:

1. **Introducción:** dotaremos del rol que tiene que usar la IA, así como información relevante general.
2. **Tarea:** describiremos qué problema debe solucionar la IA.
3. **Contexto:** daremos información sobre lenguajes, guías de estilo, ficheros, directorios, etc.
4. **Instrucciones:** especificaremos explícitamente algoritmos, estructuras de control o de datos, patrones o cualquier elemento que queramos que siga.
5. **Cierre:** formato de la salida, o cualquier información relevante para finalizar.

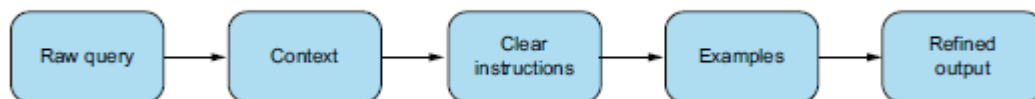




Según esta guía, pasaremos a chat GPT un prompt similar al siguiente:

1. Actúa como un experto en diseño de software.
2. Ayúdame a analizar un problema.
3. Quiero crear pruebas prácticas para obtener la licencia de radioaficionado. Generaré un banco de preguntas. Tomaré un conjunto de 35 preguntas seleccionadas al azar de ese conjunto. Quiero mostrar las posibles respuestas y seleccionar una respuesta. Al final de la prueba, me gustaría calificarla con un porcentaje.
4. Incluye detalles sobre la estructura y describe algunos posibles problemas.
5. Elabora el resultado en forma de documento de diseño de software.

El resultado se puede ver en el fichero *cap_03_01.docx*. Si ejecutamos nosotros el prompt recordar que no es determinista. Es importante recordar que este documento es una primera versión que necesitará refinamiento.

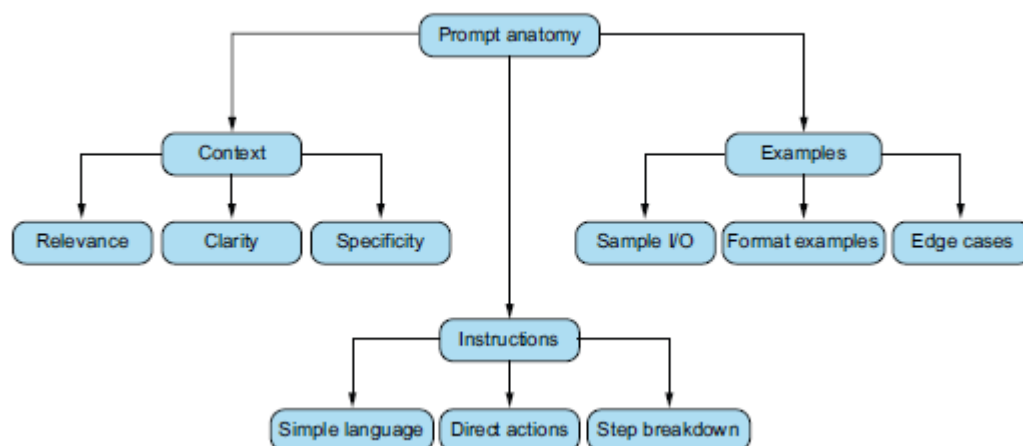


Este tipo de prompt se conoce como “role prompt”. Otros tipos son:

- **Indicaciones para la cadena de pensamiento:** pedimos a la IA que “piense paso a paso” al desglosar problemas de diseño complejos.
- **Indicaciones con pocos ejemplos:** introducimos ejemplos de cómo debe ser un buen resultado antes de solicitar la solución al problema.
- **Sugerencias de persona:** más allá de simplemente “Actúa como un experto en diseño de software”, podemos intentar: “Actúa como un arquitecto de software con 15 años de experiencia en sistemas escalables” para obtener conocimientos más específicos.

Prompt engineering

En resumen, la estructura de un buen prompt se puede resumir en la siguiente figura:



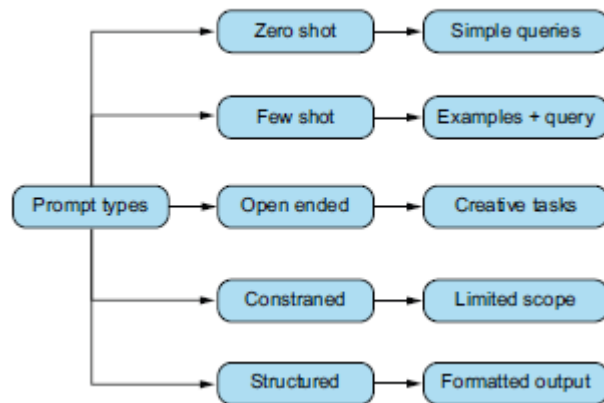
Como todo proceso de ingeniería tiene sus principios, el prompt engineering también y los podríamos resumir en:

- Definir objetivos claros. Especificar el resultado que se busca.
- Utilizar un lenguaje específico. Elegir palabras directamente relacionadas con la tarea.
- Equilibrar estructura y flexibilidad. Proporcionar suficiente estructura para guiar la respuesta, dejando al mismo tiempo margen para ideas creativas.
- Utilizar pistas contextuales. Incorporar información relevante y utilizar palabras clave que se ajusten al contenido de la tarea.
- Diseñar instrucciones claras y concisas. Evitar la ambigüedad y prioriza la claridad.
- Utilizar ejemplos de forma eficaz. Proporcionar ejemplos concretos para ilustrar el resultado deseado y guiar la respuesta del modelo.
- Emplear técnicas de juego de roles. Asigna roles o personajes específicos a la IA.
- Dividir las tareas complejas en pasos. Dividir los problemas intrincados en subtareas más pequeñas y manejables.
- Utilizar una longitud adecuada para las indicaciones. Buscar el equilibrio entre proporcionar suficiente información y evitar una sobrecarga que pueda confundir al modelo.
- Usar especificaciones de formato. Diseña claramente el formato de salida (puntos clave, fragmentos, estructura de datos, etc.).

Actúa como un ingeniero de prompts con cinco años de experiencia, especializado en optimizar las indicaciones para lograr la máxima claridad, relevancia y calidad de los resultados. Analiza la indicación que aparece a continuación para mejorar su especificidad, precisión y capacidad para generar el resultado deseado. Antes de continuar, hazme preguntas específicas para aclarar el contexto esencial, como el público al que va dirigido, el tono deseado y cualquier restricción específica. Basándote en esta información, proporciona una indicación optimizada que pueda ajustarse para casos de uso específicos o modelos de IA.

Tipos de prompts

Cada tipo de prompt se deberá usar según la respuesta deseada por el modelo. A continuación, vemos los tipos más relevantes.



- Zero shot. Permite a los modelos de lenguaje (LLM) realizar nuevas tareas para las que no han sido entrenados específicamente, utilizando los conocimientos que ya han adquirido. Deben ser básicos y concisos. A medida que los modelos vayan mejorando, las indicaciones sin datos previos serán cada vez más eficaces. A veces resultan útiles cuando no se dispone de mucha información sobre un problema, o cuando se buscan respuestas rápidas y sencillas.

JM Write a Python function that calculates the factorial of a given number.

- Few-shot prompting. Permite que los modelos LLM realicen tareas tras recibir unos pocos ejemplos.

JM You are a Python developer. Here are a few examples of functions:

1 Function to calculate the square of a number:

```
def square(x):
    return x * x
```

2 Function to reverse a string:

```
def reverse_string(s):
    return s[::-1]
```

Now write a function that calculates the factorial of a number:

```
def factorial(n):
```

- Open-ended prompts. Dada la amplia variedad de respuestas que puede suscitar, la pregunta abierta es un enfoque excelente. Puede dar lugar a resultados creativos y variados.

JM Compare and contrast Python frameworks like Django, Flask, and FastAPI for web development. In what scenarios would you choose one over the others?

- Constrained prompts. Limitan el alcance de la respuesta del modelo. Proporcionan directrices específicas basadas en los criterios que se establezcan.

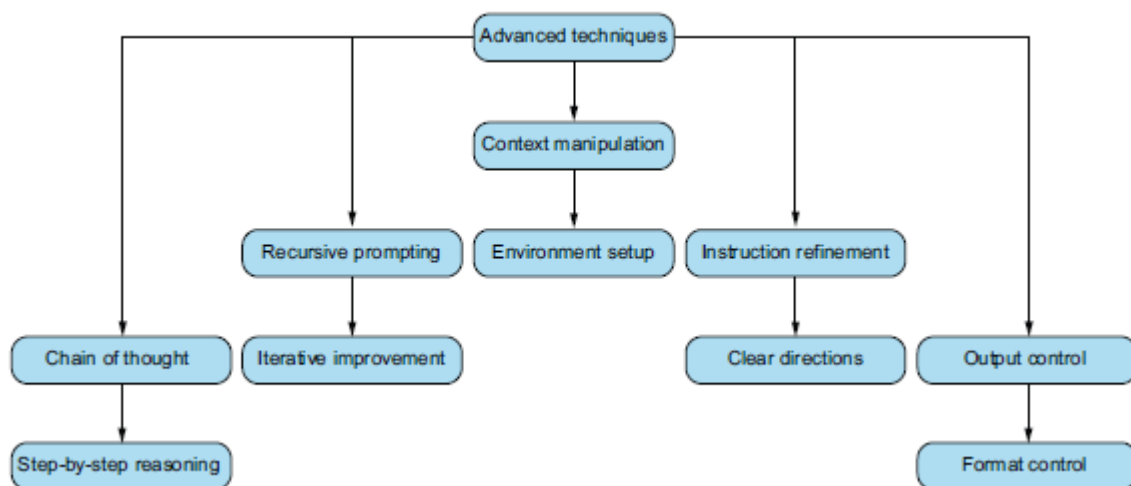
JM Write a list comprehension in Python that generates the squares of even numbers from 1 to 20. The solution should be a single line of code.

- Iterative prompts. Permite aprovechar todo el potencial de los modelos y obtener los resultados deseados refinando la respuesta en múltiples iteraciones.
- Structured prompts. Hay que ajustar el modelo a un formato predefinido.

- JM** 1 Create a comprehensive guide for optimizing Python code performance. Include the following sections:
- Profiling techniques
 - Common bottlenecks
 - Optimization strategies
 - Algorithm improvements
 - Data structure choices
 - Use of built-in functions and libraries
 - Multiprocessing and multithreading
 - Best practices and tips
- 2 Provide a structured overview of our custom Python library:
- a Introduction
 - b Core concepts
 - c Coroutines
 - d Event loops
 - e Futures and Tasks
 - f Basic usage
 - g Advanced features
 - h Real-world use cases

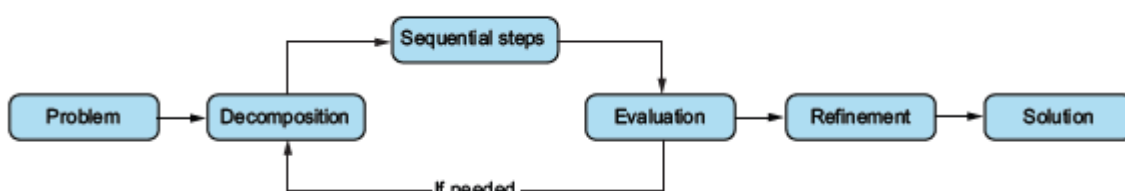
Cualquiera de las técnicas explicadas anteriormente y las siguientes se pueden combinar en cualquier momento para llegar a la solución que buscamos.

Prompts avanzados



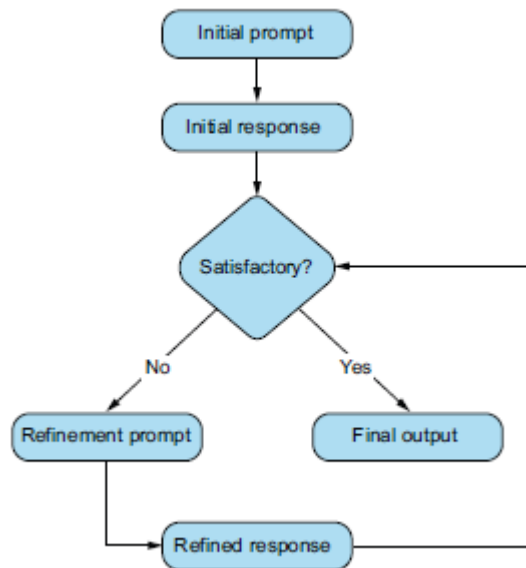
Chain-of-thought prompting

Consiste en pedir al modelo que razone paso a paso que muestre su razonamiento antes de dar la respuesta final. En otras palabras, se anima al modelo a desglosar el problema.

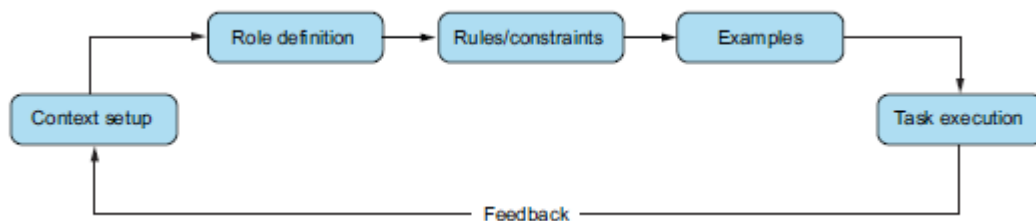


Recursive prompting

Se basa en el perfeccionamiento y la mejora iterativa de la respuesta generada por un modelo. En lugar de producir una respuesta en un solo paso, el prompt recursivo la divide en etapas.

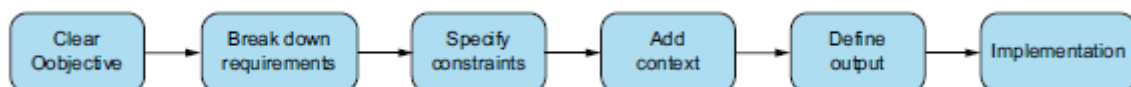


Context manipulation



Consiste en crear un entorno óptimo dentro de la solicitud para ayudar al modelo a generar respuestas precisas y relevantes. Puede resultar útil para generar código, resolver problemas o dar formato a la documentación para que se ajuste a un estilo concreto.

Instruction refinement



Proporcionar instrucciones claras y concretas para obtener mejores respuestas de los modelos de IA.

Instruction refinement



Se establecen unas directrices claras, se podrá asegurar que el resultado no solo tenga un buen aspecto, sino que también respete unas normas y formatos específicos

Los anti-prompts

O lo que deberíamos evitar a toda costa si queremos obtener buenos resultados.

- Indicaciones imprecisas. Es el clásico “No funciona, por favor, arréglalo” o “Escribe algo que haga X”.
- La solicitud sobrecargada. Se trata del problema contrario: pedirle a la IA que haga demasiadas cosas a la vez. Por ejemplo: Corrige estos 5 errores y añade también estas 3 funciones de una sola vez.
- No plantear la pregunta. A veces, los usuarios proporcionan mucha información, pero nunca formulan claramente una pregunta ni especifican lo que necesitan.
- Criterios imprecisos. A veces puede que se pida una optimización o una mejora, pero no se define en qué consiste; por ejemplo: “Haz que esta función sea más rápida”. ¿Más rápida según qué criterio?
- Ignorar las aclaraciones o los resultados de la IA. A veces, la IA puede responder con una pregunta aclaratoria o una suposición.
- Incoherencia. Si cambiamos constantemente la forma de formular la pregunta o mezclamos diferentes formatos de una sola vez, el modelo puede confundirse. Dos ejemplos son alternar entre la primera y la tercera persona en las instrucciones o mezclar pseudocódigo con código real de forma confusa.
- Referencias vagas como “el código anterior”. Al utilizar el chat, si decimos “la función anterior” o “el resultado anterior”, hay que asegurarse de que la referencia sea clara. Si la conversación es larga, la IA podría perder el hilo o elegir el fragmento de código equivocado.

Patrones que funcionan en el flujo de trabajo

Tras observar a docenas de equipos, estas son las tres pautas que se ha comprobado que funcionan de forma sistemática tanto en los flujos de trabajo individuales como en los de equipo:

- La IA como redactor inicial. El modelo de IA genera el código inicial y, a continuación, los desarrolladores lo perfeccionan, lo refactorizan y lo prueban.
- La IA como programador en pareja. El desarrollador y la IA mantienen una conversación constante, con ciclos de retroalimentación muy estrechos, revisiones frecuentes del código y un contexto mínimo.
- La IA como validador. Los desarrolladores siguen escribiendo el código inicial y, a continuación, utilizan la IA para validarlo, probarlo y mejorarlo

Ejemplos de prompts para programadores

- Especificar desde el principio los patrones arquitectónicos y los principios de diseño que se quieren seguir:

Genera una API REST utilizando el patrón de repositorio y los principios SOLID. La API debe gestionar la autenticación de usuarios con los siguientes requisitos: [...]

- Utilizar etiquetas XML para forzar una salida estructurada cuando se necesite que la aplicación analice segmentos específicos:

Genera una función JavaScript que valide direcciones de correo electrónico.
Proporcione la respuesta en estas secciones:

```
<requirements>...</requirements>
<implementation>...</implementation>
<tests>...</tests>
```

-
- Usar una indicación de inversión de roles para que el LLM diagnostique y explique los errores de codificación como si estuviera guiando a un compañero:

Eres un desarrollador sénior que explica este error a un desarrollador junior. ¿Qué lo está causando y cómo lo solucionarías?

Error: TypeError: No se puede leer la propiedad "map" de undefined...

- Desglosar tareas de programación complejas
-

Creemos un sistema de almacenamiento en caché paso a paso:

- 1 Primero, muéstrame la definición de la interfaz de caché
 - 2 A continuación, implementa una caché en memoria
 - 3 Por último, añade políticas de expulsión de la caché
-

- Utilizar la técnica de la «cadena de pensamiento» al pedir al LLM que explique su razonamiento sobre las decisiones arquitectónicas antes de escribir el código:
-

Antes de implementar el sistema de autenticación de usuarios, explica tu proceso de razonamiento a la hora de elegir entre la autenticación JWT y la basada en sesiones para nuestra arquitectura de microservicios.

- Especificar explícitamente los casos extremos y las condiciones límite en la solicitud al generar casos de prueba:

Escribe pruebas unitarias para una función que procesa restricciones de edad.

Incluye pruebas para: valores negativos, cero, edades fraccionarias, valor entero máximo y entradas nulas/indefinidas.

-
- En revisiones de código, crea una lista específica de lo que se necesita que el LLM busque:

Revisa este código en busca de:

- 1 Vulnerabilidades de seguridad
- 2 Cuellos de botella de rendimiento
- 3 Gestión de errores
- 4 Duplicación de código
- 5 Violaciones de los principios SOLID

-
- Utilizar indicaciones de pocos ejemplos con ejemplos explícitos cuando queramos código en un estilo o patrón específico:

Genera un middleware de gestión de errores siguiendo este patrón: [ejemplo 1] [ejemplo 2].

Ahora crea un middleware similar para el registro y la autenticación.

-
- Especificar las restricciones y requisitos de rendimiento desde el principio al optimizar el código:

Optimiza esta consulta de base de datos para gestionar 1000 usuarios simultáneos con un tiempo de respuesta inferior a 100 ms. Código actual: [...]

-
- Para el diseño de API, utilizar indicaciones basadas en escenarios para tener en cuenta diferentes casos de uso:

Diseña una API REST para un carrito de la compra que gestione estos escenarios: Un usuario invitado añade artículos

-
- Especificar explícitamente el público destinatario y el nivel técnico al elaborar la documentación:

Redactar la documentación de la API para este punto final dirigida a desarrolladores junior que estén familiarizados con REST, pero que no conozcan nuestro sistema de autenticación ni la autenticación en general.

-
- Utilizar preguntas comparativas para comprender las ventajas e inconvenientes de los diferentes enfoques técnicos:

Compara MongoDB y PostgreSQL para nuestro sistema de gestión de usuarios, teniendo en cuenta: la escalabilidad, la complejidad de las consultas, los requisitos de consistencia de los datos y la carga de mantenimiento.

-
- Proporcionar tanto el código actual como los “code smells” específicos que deseamos abordar para las tareas de refactorización:

Refactoriza este código para eliminar los siguientes problemas: método largo, lógica duplicada y acoplamiento. Código actual: [...]

- Solicitar alternativas de diseño y evaluar las ventajas e inconvenientes antes de aplicar un patrón específico:
 - ¿Qué patrones de diseño podrían gestionar cálculos de precios dinámicos? Explica las ventajas e inconvenientes entre Strategy, Template y Chain of Responsibility antes de implementar la mejor opción.
 - Utilizar ejemplos de “antipatrones” para comprender qué no se debe hacer en las implementaciones:

¿Cuáles son los antipatrones más comunes a la hora de implementar el almacenamiento en caché en una arquitectura de microservicios, y cómo debemos evitarlos en este código: [...]

-
- Para el código relacionado con la seguridad, solicitar explícitamente el cumplimiento de las directrices de OWASP y las mejores prácticas de seguridad:

Generar un middleware de validación de entradas de usuario que siga las directrices de seguridad de OWASP y prevenga ataques XSS, de inyección SQL y CSRF.

-
- Solicitar un análisis de complejidad temporal y espacial junto con la implementación al generar algoritmos complejos:

Implementa una solución para encontrar archivos duplicados en un árbol de directorios. Incluir un análisis Big O de la complejidad temporal y espacial, y explicar las compensaciones realizadas.

-
- Utilizar un enfoque basado en componentes para el desarrollo front-end con el fin de garantizar una arquitectura fácil de mantener:

Antes de crear el panel de control, muéstrame la jerarquía de componentes y el diagrama de flujo de datos. A continuación, desarrolla cada componente empezando por los nodos hoja.

-
- Para el diseño del esquema de la base de datos, proporcionar reglas de negocio y restricciones en un formato estructurado:

Diseña un esquema de base de datos para una plataforma de aprendizaje electrónico en la que:

- 1 Los usuarios puedan inscribirse en varios cursos
- 2 Cada curso tenga varios módulos
- 3 Se realice un seguimiento del progreso por módulo

-
- Especificar los escenarios de error exactos y las estrategias de recuperación al implementar la gestión de errores:

Implemente la lógica de reintento para este cliente API con:

1. Un máximo de 3 reintentos
2. Retraso exponencial
3. Patrón disyuntor para fallos persistentes

-
- Utilizando indicaciones de transición de estado para implementar flujos de trabajo complejos:

Crea una máquina de estados para el procesamiento de pedidos que gestione:

pendiente → pagado → en proceso → enviado → entregado,
con estados de error para cada transición.

- En caso de optimización del rendimiento, proporcionando métricas específicas y datos para perfilar:

Este punto final de la API tarda 2 s en responder. Aquí está el gráfico de llamada y el plan de ejecución de la consulta de la base de datos.

Sugiere e implementa optimizaciones para reducir el tiempo de respuesta a menos de 200 ms.

-
- Identificar claramente los requisitos específicos del entorno al generar archivos de configuración:

Genera archivos Docker Compose para los entornos de desarrollo, staging y producción.

El entorno de desarrollo debe tener recarga en caliente, el de staging debe tener monitorización y el de producción debe tener alta disponibilidad.

-
- En las implementaciones de registro, especificar los datos concretos necesarios para la observabilidad:

Implementar un registro estructurado que recoja: marca de tiempo, servicio

-
- Usar indicaciones de pruebas basadas en escenarios para generar conjuntos de pruebas exhaustivos:

Genera pruebas de integración para el módulo de procesamiento de pagos que abarquen: pagos correctos, tarjetas rechazadas, tiempos de espera de red, reembolsos parciales y escenarios de devolución de cargo.

-
- Especificación clara de los requisitos de compatibilidad con versiones anteriores para el control de versiones de la API:

Muestra cómo implementar un control de versiones de la API que:

- 1 Mantenga la compatibilidad con la v1
- 2 Introduzca cambios de compatibilidad en la v2
- 3 Proporcione documentación de migración para los clientes

-
- Utilización de indicaciones de escenarios de ataque al implementar la autenticación:

Implemente la autenticación JWT que evite: ataques de repetición de tokens, ataques de sincronización, intentos de fuerza bruta y robo de tokens a través de XSS.

-
- En la implementación de estrategias de almacenamiento en caché, especificar explícitamente las reglas de invalidación de la caché:

Implemente la gestión de la caché para los datos de productos con estas reglas:

- 1 Invalidar tras cambios de precio
 - 2 TTL de 1 hora
 - 3 Invalidación masiva durante eventos de rebajas
-

- Especificar sugerencias de tipo y preferencias de formato de cadenas de documentación para obtener un código más mantenible al generar clases de Python:

Crea una clase `User` con sugerencias de tipo utilizando las características de Python 3.10+. Incluye cadenas de documentación al estilo de Google y gestiona estos atributos: `username` (str), `login_attempts` (int), `last_login` (datetime).

- Utilizar indicaciones centradas en el mantenimiento para generar código sostenible:

Implementa esta función asumiendo que será mantenida por un equipo diferente dentro de 6 meses. Incluye: nomenclatura clara, comentarios exhaustivos, registro, supervisión y documentación.

Ejercicio

Pasemos ahora a crear el documento definitivo, partiendo del generado anteriormente, con el prompt anterior estudiamos el documento, lo iremos refinando poco a poco, **NO** cierres la ventana de Chat.

Antes de adentrarnos en este proceso, hay que tener en cuenta algunos aspectos de los documentos de diseño que pueden pasar al generarlos con la IA:

- Patrones arquitectónicos demasiado genéricos sin justificación.
- Recomendaciones tecnológicas que no tienen en cuenta las limitaciones del problema.
- Falta de gestión de errores o de consideraciones de seguridad.
- Ignorar la compatibilidad multiplataforma cuando sea fundamental para la aplicación

Revisar ahora el fichero `cap_03_01.docx` si no se ha hecho antes. Si estamos siguiendo el capítulo y hemos generado el nuestro propio, debemos asegurarnos que el punto a revisar coincide con los prompts aquí utilizados a continuación, tampoco pasaría nada si se usa otro en el que detectemos alguna deficiencia.

Uno

Hemos revisado el documento y es una buena primera aproximación, podemos criticar algunas elecciones como es Json en vez de una BBDD o alguna más, pero se puede seguir refinando. En concreto, no nos gusta el segundo punto, parece poco exhaustivo, vamos a ampliarlo.

Da más detalles en el apartado “2. Objetivos”. Profundiza en los temas principales.

Podemos ver el resultado en el fichero `cap_03_02.docx`.

Dos

En la primera revisión vimos que el documento no usaba una pila adecuada, para el almacenamiento por ejemplo usaba JSON, vamos a concretar más.

Es una aplicación web basada en Python Flask con el patrón MVC. Concreta el punto 4 Arquitectura General, adaptándolo a los nuevos requerimientos.

César San Juan Pastor

A continuación, escribimos:

Describe brevemente la pila técnica a usar

Terminamos con:

Añade a la descripción anterior lo que falte para el ciclo completo de una aplicación, hasta CI/CD

Podemos ver el resultado en el fichero **cap_03_03.docx**.

Tres

Para finalizar, generaremos historias de usuario, pero como no sabemos muy bien qué es esto, le vamos a preguntar a Microsoft Copilot.

Qué son las user stories en la documentación de diseño de una aplicación web

La respuesta es bastante extensa, pero nos quedamos con la primera parte que nos sirve para hacernos una idea: *Una user story (historia de usuario) en la documentación de diseño de una aplicación web es una descripción breve, simple y centrada en el usuario que explica qué necesita hacer un usuario, para qué lo necesita y qué valor obtiene. Es uno de los elementos fundamentales en metodologías ágiles como Scrum y se utiliza para guiar el diseño y desarrollo del producto.*

El prompt que damos a ChatGPT:

Actúa como un Product Owner con experiencia. Crea historias de usuario para este proyecto de software.

Podemos ver el resultado en el fichero **cap_03_04.docx**.

El documento final, integrando todos los cambios, nos da una base sólida para poder empezar, pero se puede seguir refinando hasta que deseemos. Podríamos incluir información similar a:

- Crea una plantilla de especificaciones técnicas para una API REST.
- Genera un esquema para un documento de diseño de esquema de base de datos.
- Crea una plantilla de registro de decisiones de arquitectura.
- Cambia la base de datos a MongoDB.
- Etc.

El punto principal, por si no te habías dado cuenta, es el contexto, hemos realizado todo el ejercicio en una única sesión, no hemos cerrado para nada la ventana de Chat, permitiéndonos esto refinar de forma continuada.

Práctica

Elige una aplicación de tu interés, realiza el mismo proceso que hemos hecho en este punto, pero usando otro Agente.

Nosotros hemos hecho el mismo ejercicio, pero con Microsoft Copilot, se pueden comparar los resultados, los ficheros son los mismos, pero añaden el sufijo: **MS_Copilot** al mismo para diferenciarlos.

Práctica

1. Crea un único documento de ChtGPT con lo generado en este capítulo.
2. Crea un único documento de MS Copilot con lo generado en este capítulo.
3. Pídele a un tercer Agente, que no sea ninguno de los dos usados que los analice y genere las fortalezas y debilidades de cada uno de los ficheros.
4. Genera un informe detallado de tus impresiones reales al comparar ambos documentos y ratificar con lo generado en el punto 3.

Apreciación personal: La generación de documentación para las fases de análisis y diseño es una función muy atractiva, pero como hemos visto hay que limitarla, depurarla y refinarla. Para pequeños proyectos podemos utilizarla sin más, para integrarla en grandes proyectos hay que restringir el ámbito de las respuestas, los prompts deben ser muy concretos, para una única tarea, no tan generales como aquí.

Tema 4 – Generación del esqueleto

Introducción

En este capítulo abordaremos la generación de código. Después de ver los cimientos y crear documentación de inicio, hay que escribir el código. Vamos a ello.

Crear software es como construir una casa. Los cimientos es el primer paso y el que sustenta el resto, no podemos empezar sin ellos. Además, construir el resto de la casa será muy complicado si no cumplen los requisitos.

Crear toda la aplicación completa de una vez es imposible, por lo que haremos uso de un concepto muy extendido en la creación de software: stub. El “stubbing” consiste en crear un fragmento de código simplificado, conocido como “stub”, que actúa como sustituto del código funcional. Una vez creados varios stubs, se pueden conectar y ejecutar la aplicación, creando un prototipo. El “stub” proporciona un mecanismo para probar la estructura del proyecto y el flujo de trabajo con ella rápidamente, sin tener que profundizar en los detalles individuales de cada parte. Veamos lo que sería un ejemplo de código.



```
class DatabaseManager:
    def __init__(self):
        pass
    def connect_to_database(self):
        # TODO: Implement this method to connect to the database.
        print("Connecting to the database...") # Placeholder
        return True # Simulate successful connection
    def fetch_data(self, query):
        # TODO: Implement this method to fetch data from the database.
        print("Fetching data...") # Placeholder
        return [] # Simulate empty result set
```

Si examinamos el código detenidamente, comprobamos que realmente no hace nada, imprime textos y devuelve datos simulados. Esto son stubs. La ventaja que nos proporcionan es que podemos irlos sustituyendo poco a poco por código funcional. En terminología más común, podríamos decir que es la creación del “esqueleto” de la aplicación. Para poder generar el esqueleto de una aplicación de forma eficiente, tendremos en cuenta lo siguiente:

- Crear implementaciones preliminares de una función antes de implementarla.
- Utilizar la IA generativa para que sugiera firmas de métodos y estructuras de clases.
- Empezar con funciones vacías para establecer interfaces entre componentes.
- Considerar la posibilidad de implementar una función cada vez.
- Utilizar notas de “por hacer” (TODOs) para documentar la finalidad de cada stub.
- Asegurar que la aplicación con stubs se ejecuta antes de añadir nuevas funcionalidades.
- Recordar el principio YAGNI (you aren’t gonna need it, “no lo vas a necesitar”): no crear stubs para funciones que quizá no se implementen.

Conseguir los requerimientos funcionales

Las herramientas basadas en IA son muy buenas para conseguir requerimientos y generan una solución bastante aceptable. Pero si recordamos de los temas anteriores, cuando trabajamos con IA,

lo más importante es el contexto y el prompt, vamos a dar guías de creación de este prompt así como el procedimiento para llegar a buenos resultados.

El contexto está claro, el documento del tema anterior. Al utilizar la IA para extraer requisitos:

- Empezar con indicaciones claras y específicas que definan el alcance.
- Solicitar los resultados estructurados (por ejemplo, requisitos numerados, listas categorizadas).
- Pedir que se identifiquen posibles dificultades para cada requisito.
- Utilizar indicaciones de seguimiento para profundizar en áreas específicas.
- Validar siempre los resultados con las partes interesadas o los expertos en la materia.
- Recordar que la IA puede “alucinar” requisitos que parecen plausibles pero que en realidad no son necesarios.

El proceso podría ser algo de este estilo:

- Solicitar los requisitos clave.
- Convertirlos en historias de usuario.
- Crear esbozos basados en esas historias.

¿Cuáles son los requisitos clave que podemos extraer del documento a continuación escrito de diseño de software?

El documento generado se puede ver en **cap_04_01.docx**. Como es muy extenso lo vamos a reducir.

Saca un listado de los RF exclusivamente, con el nombre, número y descripción breve.

ID	Nombre	Descripción breve
RF-01	Gestión del Banco de Preguntas	El sistema debe cargar, almacenar y administrar un banco de preguntas con identificador único, opciones de respuesta y respuesta correcta.
RF-02	Generación de Exámenes	El sistema debe generar automáticamente una sesión de examen compuesta por exactamente 35 preguntas seleccionadas aleatoriamente y sin repetición.
RF-03	Presentación de Preguntas	El sistema debe mostrar una pregunta por pantalla junto con todas sus opciones de respuesta e indicar el progreso del examen.
RF-04	Registro de Respuestas	El sistema debe registrar y conservar las respuestas seleccionadas por el usuario, permitiendo su modificación antes de finalizar el examen.
RF-05	Navegación del Examen	El sistema debe permitir avanzar, retroceder, saltar preguntas y visualizar el estado de las preguntas durante la sesión.
RF-06	Finalización del Examen	El sistema debe permitir finalizar el examen de forma controlada, solicitando confirmación e informando preguntas sin responder antes del envío.
RF-07	Calificación Automática	El sistema debe evaluar automáticamente las respuestas del usuario, calcular la cantidad de aciertos y errores, y obtener el porcentaje final.

ID	Nombre	Descripción breve
RF-08	Presentación de Resultados	El sistema debe mostrar un resumen del desempeño del usuario, incluyendo resultados generales y, opcionalmente, el detalle por pregunta.

Es recomendable presentar estos resultados en una reunión de validación y verificar los datos. Una breve reunión asegurará que vamos por buen camino, ahorrará muchos quebraderos de cabeza más adelante, **la revisión humana es imprescindible**.

Se trata de un proyecto de gran envergadura, por lo que merece la pena dedicar tiempo a explorar más opciones. Podemos usar Gemini, de Google, para comparar (los resultados están en el directorio de fuentes del capítulo). Siempre es una muy buena idea hacer referencias cruzadas con diferentes herramientas y después generar un documento propio a revisar en la reunión comentada anteriormente.

Es importante revisar el documento generado haciendo lo siguiente:

- Comparar con la documentación original del proyecto.
- Validar con las partes interesadas, especialmente en el caso de los requisitos técnicos.
- Buscar características “alucinadas” (cosas que parecen plausibles pero que no figuraban en el plan original).
- Comprobar la coherencia entre las diferentes secciones.
- Asegurarse que todos los requisitos sean comprobables y específicos.
- Desconfiar de los requisitos excesivamente genéricos que no aportan valor.
- Tener en cuenta cómo afecta cada requisito al calendario de desarrollo.

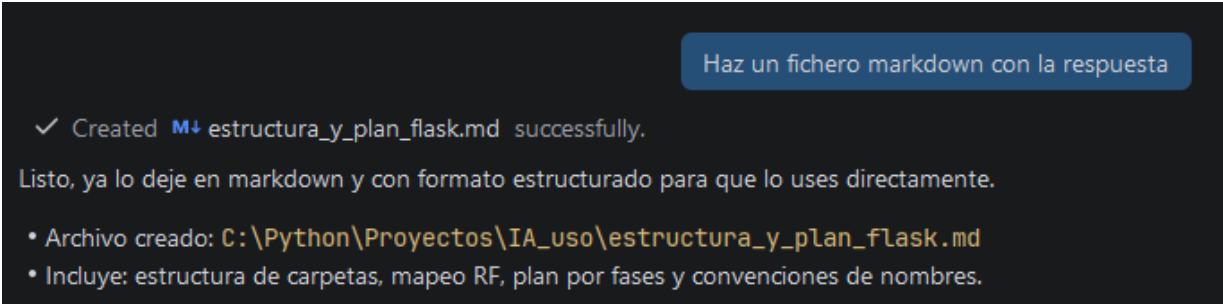
Creando el esqueleto de la Aplicación

Abriendo la ventana chat de GitHub Copilot y en el modo Agente introducimos el siguiente prompt.

Esta será una aplicación Python Flask. Muestra las mejores sugerencias de estructura de carpetas y nombres de archivos para la aplicación de los requisitos funcionales descritos en {{{RF}}}, crea un plan de aplicación. Genera un fichero Markdown con la respuesta.

{{{RF}}}

Aquí copiamos los RF...

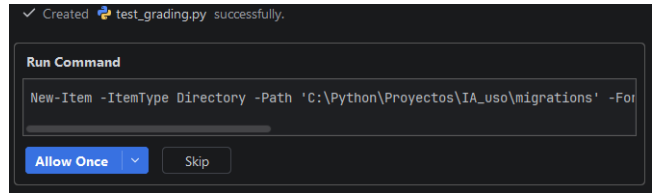
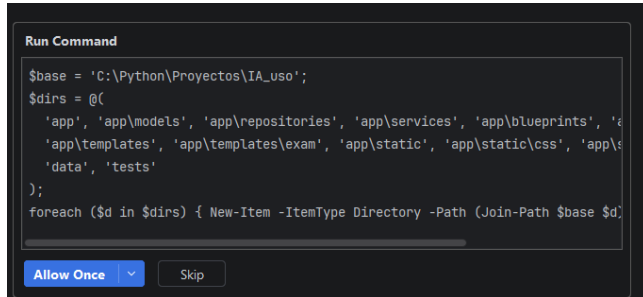
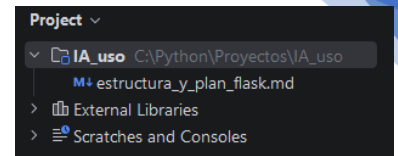


Tras la segunda revisión

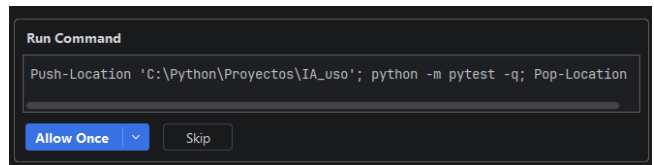
Genera el esqueleto inicial completo de esta estructura (archivos base + modelos + rutas vacías + tests mínimos) listo para arrancar con Flask

Estructura del proyecto antes.

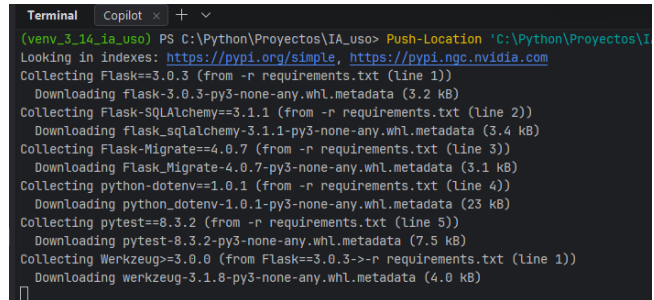
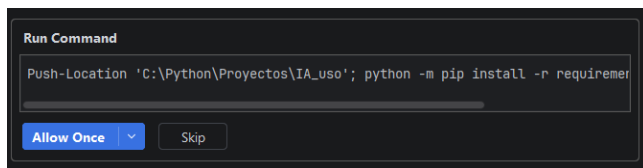
Nos va pidiendo permiso para todo lo que va a implementar, directorios, ficheros, etc.



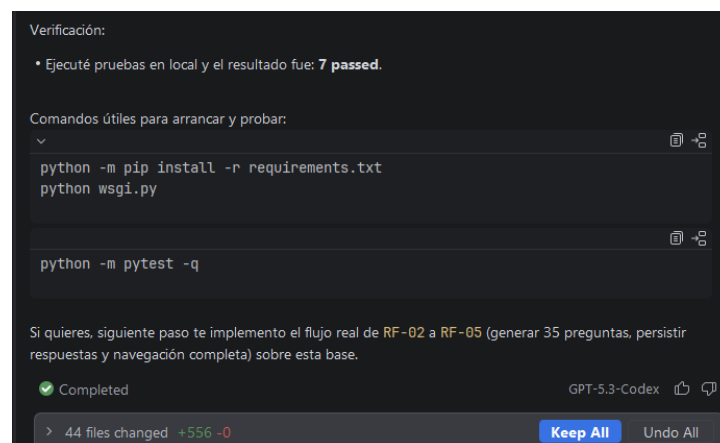
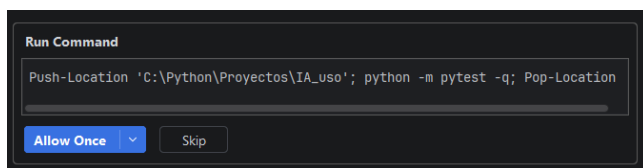
Creación de Test.



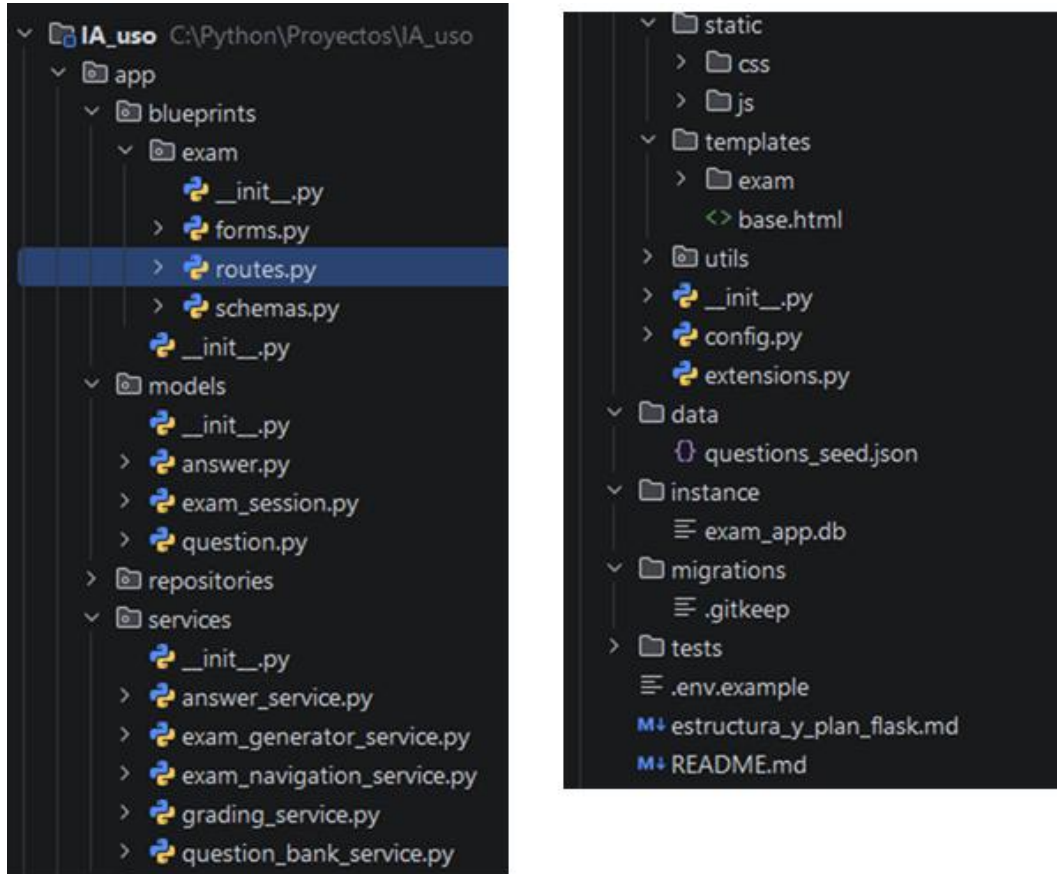
Instalación de los requerimientos de Python.



Ejecución de test.



Estructura final de ficheros y directorios.



Lanza la aplicación

Run Command

Push-Location 'C:\Python\Proyectos\IA_uso'; python wsgi.py

Allow Once

Skip

✓ Ran get_terminal_output tool

He iniciado la aplicación en segundo plano y te dejo cómo verificarla rápido.

- Proceso lanzado en terminal background con id: **fbd5b44e-34d5-4c93-899c-2610984d52f5**
- Punto de acceso esperado (por configuración actual de **wsgi.py**): **http://127.0.0.1:5000**

Si quieres, ahora mismo puedo comprobar logs del proceso y confirmar que ya está escuchando correctamente.



Hay que tener en cuenta estas limitaciones de la IA cuando se utilice para el diseño de software:

- La IA puede sugerir tecnologías populares pero inadecuadas.
- Las sugerencias técnicas requieren evaluación crítica.
- Los modelos tienden a recomendar lo que han visto con más frecuencia, no necesariamente lo que es mejor.
- La IA no comprende las habilidades ni las limitaciones específicas del equipo.
- Los modelos pueden mostrarse demasiado seguros de sus recomendaciones.
- Hay que comprobar siempre las sugerencias de la IA con los requisitos reales.
- Recordar que nosotros somos los diseñadores: la IA es solo una herramienta.

También miraremos los siguientes principios al utilizar la IA para el diseño de software:

- Las herramientas de IA son compañeros de tormenta de ideas, no arquitectos.
- El mejor uso que se le puede dar es potenciar la creatividad, no sustituirla.
- Aplicar siempre los conocimientos y experiencia en el ámbito que se tienen.
- Utilizar la IA para agilizar las partes rutinarias del diseño, liberando tiempo para los problemas complejos.
- Elegir qué parte de las sugerencias de la IA conservar en función de las necesidades específicas del proyecto.
- Combinar ideas de varias sesiones de IA para obtener resultados óptimos.
- Recordar que las decisiones finales las tomamos nosotros: la IA es solo una fuente de información.

Ejercicio

Uno

Se ha generado muchísimo código en diferentes directorios y ficheros, ahora hay que revisarlo todo y comprobar que se ajusta a lo deseado. **Primero sin utilizar nada de IA:**

- Hacer un listado exhaustivo de todos los ficheros y directorios, explicar brevemente la intención de los mismos.
- Hacer un diagrama de clases.

- Estudiar la arquitectura. Describir los ficheros involucrados y los nombres de las clases en el patrón MVC.
- Estudiar la capa de datos, cómo y dónde se cargan las preguntas.
- Documentar las plantillas utilizadas y los endpoints definidos.
 - Hacer pruebas manuales para todos los endpoints
- Investigar el idioma de los nombres en variables, métodos, clases, ficheros, etc.

Dos

Usando GitHub Copilot, crear.

- La documentación de cada fichero en formato Google doc.
- Crear un documento base que explique la arquitectura creada.
- Crear un documento explicativo de la secuencia típica de uso por parte del usuario con los ficheros utilizados y la secuencia de clases, objetos y métodos llamados.
- Crear el esquema Entidad - Relación correspondiente.
- Investigar el uso de SQLAlchemy.
- Comprobar si cumple todos los requerimientos funcionales del documento usado para crear la estructura.

Tercero

Para finalizar generar un plan de desarrollo

Crea un fichero en formato markdown en el que aparezca un checklist para cumplir las fases del plan de aplicación sugerido del fichero `#file:estructura_y_plan_flask.md`

Resultados en el fichero `checklist_fases_plan_aplicacion.md`.

Generamos estimaciones de tiempo

Haz una segunda versión, pero con tiempos esperados para implementarlo para un programador junior con dos años de experiencia y uno senior con cuatro años de experiencia

Resultado en el fichero `checklist_fases_plan_aplicacion_estimaciones.md`.

Apreciación personal: Si estudiamos el código generado, es posible que tengamos que entrar ahora en una fase de refinamiento. Por ejemplo, usar SQLAlchemy para un proyecto tan simple o la estructura es excesivamente compleja para mi gusto. Los nombres asignados a directorios y ficheros, así como las clases deberían revisarse. En definitiva, es un primer paso. Yo preferiría la creación paso a paso en vez de hacer todo el esqueleto, empezar por el modelo, pasar luego a la carga de datos test, después hacer los servicios para terminar con las vistas, guiando mucho más a la IA. Pero como un ejercicio de aprendizaje puede valer.

Tema 5 – Generación de código base

Introducción

Vamos a empezar a generar código para la aplicación. Partimos del fichero `checklist_fases_plan_aplicacion_estimaciones.md` en el que creamos una lista ordenada de los pasos a realizar.

El primer paso antes de nada es crear la BBDD y hacer la carga, para ello tenemos un fichero `questions_seed_dw.csv` que vamos a utilizar, pero tiene una estructura diferente, por lo que hay que adaptar lo generado hasta ahora al nuevo formato. Lanzamos los test y vemos que falla uno.

```
FAILED [100%]
tests\test_question_bank.py:5 (test_seed_file_has_minimum_schema)
def test_seed_file_has_minimum_schema():
    seed_path = Path("data/questions_seed.json")
    > payload = json.loads(seed_path.read_text(encoding="utf-8"))
    AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

Fase 2 - Banco de preguntas (RF-01)

- Implementar carga inicial desde `data/questions_seed.json`.
 - Junior: 4-6 h
 - Senior: 2-3 h
- Agregar CRUD básico del banco (interno/admin).
 - Junior: 8-14 h
 - Senior: 4-8 h
- Validar unicidad de identificador y formato de opciones.
 - Junior: 4-8 h
 - Senior: 2-4 h

Pedimos una pequeña ayuda a la IA, pero esta vez desde el chat inline, no hace falta una ventana completa, es un cambio mínimo que deberíamos ser capaces de hacerlo nosotros sin ayuda ninguna.

Refactoriza esta función para que lea los datos de `#questions_seed_dw.csv`, cambiando el formato a csv, y cambiando la línea assert para los nuevos campos

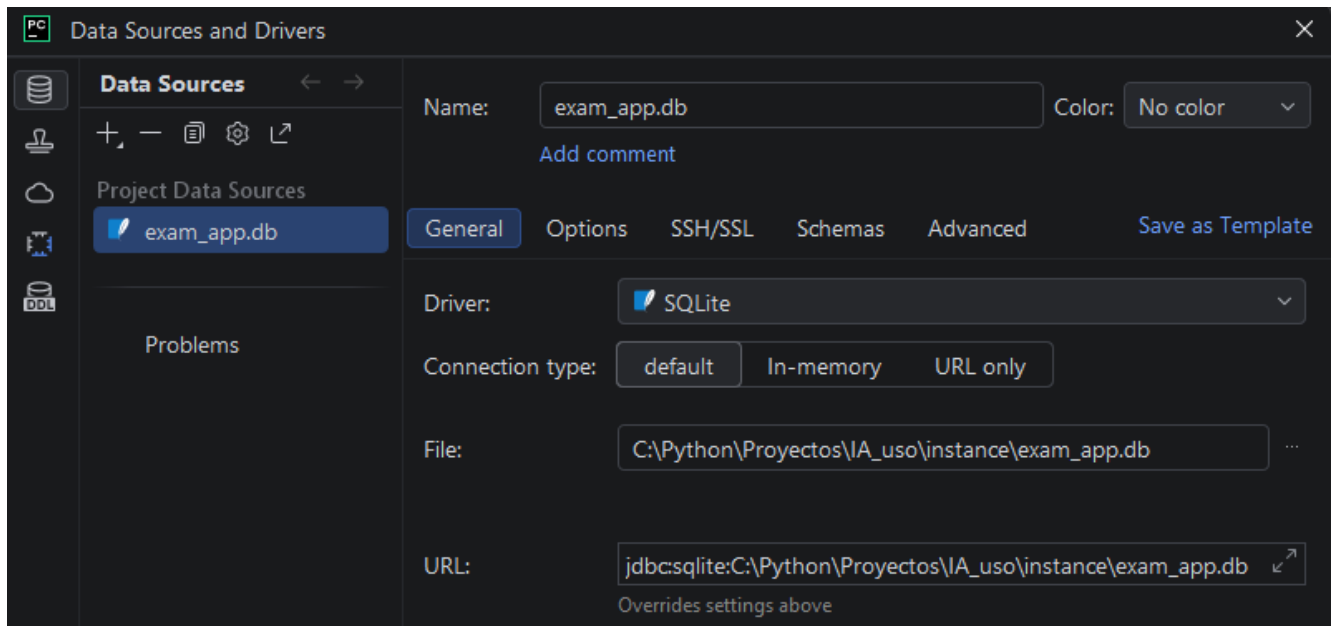
```
def test_seed_file_has_minimum_schema():
    seed_path = Path("data/general-2023-2027.csv")
    payload = json.loads(seed_path.read_text(encoding="utf-8"))
    seed_path = Path("data/questions_seed_dw.csv")
    with seed_path.open(encoding="utf-8", newline="") as seed_file:
        payload = list(csv.DictReader(seed_file))

    assert isinstance(payload, list)
    assert len(payload) >= 1
    first = payload[0]
    assert {"id", "refs", "question", "answers", "figure", "correct_letter"}.issubset(first.keys())
    assert {
        "id",
        "refs",
        "question",
        "answer_a",
        "answer_b",
        "answer_c",
        "answer_d",
        "figure",
        "correct_letter",
    }.issubset(first.keys())
```

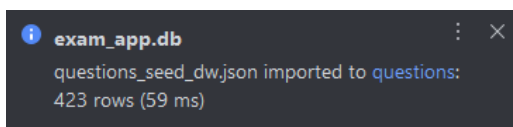
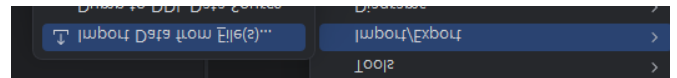
Con los cambios aceptados, comprobamos el código generado y vemos que falta un campo del fichero csv, **correct**, que añadimos nosotros a mano y los nombres de las preguntas no concuerdan.

El formato del archivo es csv, desde el IDE podremos importarlo a la BBDD directamente pero antes hay que crear una conexión SQLite con el fichero **instance/exam_app.db**.

```
===== 7 passed in 0.08s =====
Process finished with exit code 0
```



Con todo configurado podemos importar el fichero CSV, botón derecho sobre la conexión abierta, importar fichero.



Carga completa de 423 preguntas. Hay que modificarla para añadir la clave primaria. Se puede modificar la tabla una vez creada, utilizando la categoría *keys* para crearla sobre el campo id.

Fase 2. Modelo y conexión.

Primeros datos

Una vez creada y precargada la BBDD vamos a adaptar el modelo que se creó con el esqueleto, le pedimos a la IA que nos sugiera cambios.

Examina la BBDD SQLite que está en `#instance/exam_app.db`, la tabla `questions` y sugiere cambios en el modelo que reside en el fichero `#models/question.py`

Tras leer de la BBDD la estructura, preguntarnos varias veces si permitimos ejecutar código para leer los ficheros y los datos relacionados, sugiere los cambios adecuados en dos ficheros, aceptamos los mismos. Por simplicidad solo muestro los sugeridos en el fichero `model/answer.py`.

```
exam_session_id = db.Column(db.Integer, db.ForeignKey("exam_sessions.id"), nullable=False)
question_id = db.Column(db.Integer, db.ForeignKey("questions.id"), nullable=False)
question_id = db.Column(db.String(50), db.ForeignKey("questions.id"), nullable=False)
```

Ejecutamos los test y pasan correctamente. Si profundizamos en el código y los cambios, vemos que lo sugerido minimiza los cambios internos, ya que la estructura anterior de tabla la sigue manteniendo, no ha modificado los nombres por los que nos referiremos en la aplicación a los campos. Por ejemplo,

los nombres de las opciones no se alinean con la BBDD, habría que comprobar si esto se corresponde con nuestras restricciones. Tenemos el modelo y la BBDD sincronizada, creemos la migración inicial.

Crear la migración inicial con el contenido de la base de datos que reside en `#file:instance` y deja el fichero en el directorio migrations, génalo en SQL

Sigamos con la BBDD, conectemos el modelo a la BBDD real. Vamos a ver qué nos sugiere la IA de cómo hacerlo.

Actúa como un programador senior, examina el proyecto, sugiere varias formas de crear la conexión con la BBDD sqllite que reside en `exam_app.db`, incluye el uso de las constantes que están en `#file:config.py`

Sugiere hasta cinco opciones diferentes, nos decantamos por la tercera.

3) Configurar la URI en el factory usando `app.instance_path`

Buena opción cuando quieres que Flask controle explícitamente carpeta `instance`.

```
# app/__init__.py
from pathlib import Path
from flask import Flask
from app.config import Config
from app.extensions import db, migrate

def create_app(config_object=Config):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_object)

    if app.config.get("SQLALCHEMY_DATABASE_URI", "").startswith("sqlite:///exam_app.db"):
        db_path = Path(app.instance_path) / "exam_app.db"
        app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:/// {db_path.as_posix()}"

    db.init_app(app)
    migrate.init_app(app, db)
    return app
```

- Muy claro para equipos: "la DB de dev vive en `instance`".
- Evita sorpresas por cwd.

Parece que todo va bien, ejecutamos los test y los pasa, vamos a lanzar nuestra aplicación. Ups

```
* Restarting with stat
Traceback (most recent call last):
  File "C:\Python\Proyectos\IA_uso\wsgi.py", line 3, in <module>
    app = create_app()
  File "C:\Python\Proyectos\IA_uso\app\__init__.py", line 19, in create_app
    db.init_app(app)
```

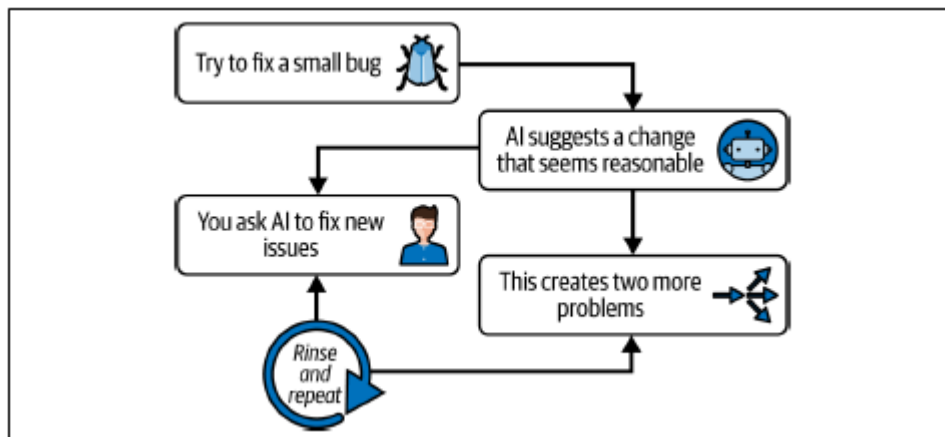
Hay algo mal, leyendo el código parece que tiene que ver con la URL de SQLite, depuremos.

Actúa como un tester con cuatro años de experiencia, lanza la aplicación, céntrate en los errores de conexión con la base de datos, en concreto la sintáxis de la url, sugiere problemas en @app/_init.py el error producido es raise exc.ArgumentError(

"Could not parse SQLAlchemy URL from given URL string"

)

Después de veinte minutos, seguimos igual, este es el principal problema que nos podemos encontrar, que parece que da soluciones, pero no es así. Esto es un típico fallo de uso.



Actúa como un programador senior con 4 años de experiencia y modifica el proyecto para integrar lanzar la migración inicial del fichero #file:001_initial_from_instance.sql al comienzo de la aplicación solo si no existe la tabla questions

Después del código generamos las pruebas necesarias

Crea un test de prueba para comprobar la inicialización de la base de datos si no existe, prueba solo la función _run_initial_sql_migration_if_needed, deja el fichero en el directorio tests

Estaría bien refactorizar el código sacando la migración del fichero init.

Refactoriza el código del fichero #file:__init__.py cambiando de lugar la función _run_initial_sql_migration_if_needed y lo necesario, déjala en un fichero utils.py en el directorio app

Este último paso es muy interesante, ya que le he forzado a que no pueda hacerlo, ya que existe en la estructura de directorios uno llamado **utils**. El Agente lo detecta y nos sugiere que va a crear un fichero llamado `migrations.py` dentro del directorio `utils` y refactorizará ahí el código.

Vamos a mostrar las diez preguntas primeras en la página principal. Intentaremos un prompt un poco más complejo, recomendando dividirlo en pasos individuales para que funcione mejor.

Modifica el endpoint `start_exam` del fichero `#file:routes.py` para que cree y llame a una función `{{1}}` del servicio que está en `#file:question_bank_service.py` para cargar un número de preguntas que se pasen por un parámetro. Después crea una función en el `#file:question_repository.py` que sea llamada desde la función `{{1}}` con la misma funcionalidad pero que devuelva las `n` primeras preguntas de la `bbdd`. Para finalizar modifica la plantilla para que muestre 10 preguntas recogiendo del `#file:question_bank_service.py`

Inicio del Examen

Se muestran las primeras 10 preguntas disponibles.

1. **G0A01** - What is one way that RF energy can affect human body tissue?
2. **G0A02** - Which of the following is used to determine RF exposure from a transmitted signal?
3. **G0A03** - How can you determine that your station complies with FCC RF exposure regulations?
4. **G0A04** - What does "time averaging" mean when evaluating RF radiation exposure?
5. **G0A05** - What must you do if an evaluation of your station shows that the RF energy radiated by your station exceeds permissible limits for possible human absorption?
6. **G0A06** - What must you do if your station fails to meet the FCC RF exposure exemption criteria?
7. **G0A07** - What is the effect of modulation duty cycle on RF exposure?
8. **G0A08** - Which of the following steps must an amateur operator take to ensure compliance with RF safety regulations?
9. **G0A09** - What type of instrument can be used to accurately measure an RF field strength?
10. **G0A10** - What should be done if evaluation shows that a neighbor might experience more than the allowable limit of RF exposure from the main lobe of a directional antenna?

No nos olvidemos de los test.

Crea test para las dos funciones nuevas, en `#file:question_repository.py` y `#file:question_bank_service.py`, deja los archivos en test. Prueba el caso límite cuando se pida cero y una, también.

Pero también hacemos pruebas manuales, y cambiamos en el endpoint 10 por 100 para ver qué pasa, y resulta que siguen apareciendo solo 10. Investigamos un poco, abrimos el fichero `start.html` y vemos que en el bucle también lo limitaba a 10, lo quitamos (ver `start_00.html` para salida generada). También añadimos a mano una variable para que se muestre el número de preguntas: `len_questions=len(questions)`. Cuando trabajamos con la IA hay que equilibrar el desarrollo del MVP con las buenas prácticas:

- Al acelerar el desarrollo con herramientas de IA, se empieza por la funcionalidad básica: primero en las características imprescindibles.
- No saltar el diseño de la base de datos: incluso en los MVP, debe existir un diseño adecuado del esquema.
- Es mejor aplazar la optimización: primero buscamos un código que funcione y las mejoras de rendimiento se harán más adelante. Este punto puede ser muy discutible, ya que en los proyectos siempre se va falto de tiempo y el después puede que no llegue nunca, generando deuda técnica.
- Simplificar siempre que sea posible: evitar la proliferación de funciones y la sobrediseño.

- Mantener la separación de responsabilidades: incluso el desarrollo rápido se beneficia de una arquitectura limpia.
- Tener en cuenta la seguridad: algunas buenas prácticas no deben sacrificarse ni siquiera en aras de la velocidad.
- Documentar.
- Probar las rutas críticas: Asegurarse de que la funcionalidad básica funciona según lo esperado.

El punto óptimo es crear código mantenible rápidamente sin generar deuda técnica que ralentice el desarrollo futuro. En resumen, y como recordatorio de los temas anteriores, para obtener un código de mejor calidad con las herramientas de IA:

- Ser específico en cuanto al lenguaje, los marcos de trabajo y las versiones.
- Incluir información sobre la estructura y los patrones del código actual.
- Especificar exactamente qué debe hacer y qué debe devolver la función.
- Mencionar las expectativas en cuanto al manejo de errores.
- Incluir rutas de archivos y detalles de la base de datos cuando sea relevante.
- Dividir las solicitudes complejas en indicaciones más pequeñas y específicas.
- Para tareas más complejas, utilizar la interfaz de chat para mantener el contexto de la conversación.

A continuación, varios ejemplos para poder diferenciar la calidad de prompts:

- Deficiente: “Conéctate a una base de datos”.
- Mejor: “Conéctate a una base de datos SQLite. El nombre del archivo es data/questions.db”.
- Óptimo: “Crea un método que se conecte a una base de datos SQLite en data/questions.db, gestione los errores de conexión y devuelva un objeto cursor”.

Pero como siempre, lo mejor es el juicio personal y desconfiar cuando:

- El código generado no respeta los principios de separación de responsabilidades.
- Se sugiere una solución compleja cuando bastaría con una más sencilla.
- La arquitectura sugerida no se ajusta a los patrones del proyecto.
- No se tienen en cuenta las consideraciones de rendimiento (p. ej., SELECT *).
- Se ignoran las mejores prácticas de seguridad.
- La IA está haciendo suposiciones sobre el proyecto que no son precisas.
- El código funciona, pero no es mantenible a largo plazo.
- Ves elecciones de bibliotecas o dependencias que no encajan con la pila de desarrollo.

Y si tenemos tiempo, hay que intentar dejar el código lo más limpio posible, generalmente buscaremos los siguientes factores para hacerlo:

- Separación de responsabilidades (acceso a la base de datos, lógica de negocio, presentación).
- Eliminación de valores codificados y adición de opciones de configuración.
- Mejora del manejo de errores y de los casos extremos.
- Facilitar la comprobación del código mediante la inyección de dependencias.
- Asegurar que el código siga los patrones y estándares específicos del proyecto.
- Eliminación de secciones de código no utilizadas o redundantes.
- Mejorar la nomenclatura de variables y funciones para mayor claridad.

Y nos centraremos en las siguientes tareas que si no se expresan de forma explícita en el prompt no se suelen incluir:

- El manejo de excepciones.
- La facilidad de prueba.
- La gestión de recursos (conexiones, manejadores de archivos).
- La configurabilidad.

Preguntas desordenadas únicas

Modifica la función `first_n_questions` de `#file:question_repository.py` para que la elección sea de `n` cuestiones aleatorias todas diferentes, modifica y usa si es necesario la función `pick_unique` de `#file:randomizer.py`

Una única línea, que bien, este es el código generado:

```
@staticmethod
def first_n_questions(limit):
    questions = Question.query.all()
    return pick_unique(questions, limit)
```

En principio, funciona, no hay problemas y cada vez hace un resultado diferente.

Inicio del Examen

Se muestran las primeras 10 preguntas disponibles

Inicio del Examen

Se muestran las primeras 10 preguntas disponibles

1. **G2E05** - What is the standard sideband frequency for voice transmissions on the 10-meter band?
2. **G8C14** - Which of the following describes a voice segment that is available to them?
3. **G1A11** - When General class licensees are operating on the 10-meter band, what is the maximum power level that they may use?
4. **G1C01** - What is the maximum transmitter power for a General class licensee operating on the 10-meter band?
5. **G5C09** - What is the capacitance of three 10-microfarad capacitors connected in parallel?
6. **G5C14** - Which of the following components is used to store energy in an electric field?
7. **G3A11** - How long does it take a coronal mass ejection (CME) to travel from the sun to Earth?
8. **G8B03** - What is another term for the magnetic field of a magnet?
9. **G2A11** - Generally, who should respond to a distress call?
10. **G4E10** - Why should a series diode be connected to the panel of a power supply?

1. **G4D08** - What frequency range is occupied by the 10-meter band?
2. **G8A06** - Which of the following is characteristic of a voice signal?
3. **G4B07** - What signals are used to conduct a RACES training drill?
4. **G2B11** - How often may RACES training drills be conducted?
5. **G1E02** - When may a 10-meter repeater retransmit the signal of a station?
6. **G5C08** - What is the equivalent capacitance of three 10-microfarad capacitors connected in parallel?
7. **G4E11** - What precaution should be taken when working with a power supply?
8. **G5B07** - What value of an AC signal produces the same heating effect as a DC signal of the same value?
9. **G3A14** - How is long distance radio communication possible?
10. **G8C15** - What does an FT8 signal report of?

Ya intuimos que algo falla. Repasemos el código, en concreto la línea: `questions = Question.query.all()`. En esta primera fase no nos debería llamar la atención, pero qué pasa si pretendemos poner la aplicación a disposición del público en internet y migramos a un servidor de BBDD más potente, y el número de preguntas creciera hasta 1 millón. En cada preparación del examen se traería toda la tabla completa, un problema de rendimiento tremendo, aquí estamos haciendo un **"SELECT * FROM Questions"**, mejor no pensarlo, anotemos un comentario de deuda técnica aquí. Cambiemos al menos el `*` y traigamos solo el id, que es lo que necesitamos.

Cambia la función `first_n_questions` para que recoja solo la clave primaria, modifica la plantilla en `#file:start.html` para reflejar el cambio.

Inicio del Examen

Se muestran 10 claves primarias de preguntas disponibles.

1. **G2E13**
2. **G4A12**
3. **G0B04**
4. **G4E04**
5. **G4B03**
6. **G7B04**
7. **G1A09**
8. **G6B01**
9. **G3A14**
10. **G2D02**

Tras los cambios sugeridos, también modifica los ficheros de test, aparecen solo los ids de las preguntas, ya no tenemos todo, se mejora algo la deuda técnica, pero se siguen trayendo todos los identificadores.

Fase 3. Modelo y conexión.

Sesión con 35 preguntas

Tras ejecutar los test, deberían aparecer las tablas necesarias, vamos a crear el script de migración inicial

Crear la migración con el contenido de la base de datos que reside en `#file:instance` solo para las tablas `exam_sessions` y `answers` y deja el fichero en el directorio `migrations`, génalo en SQL

Añadimos al comienzo del script SQL el borrado de las tablas ya que no lo ha hecho y queremos dejar la BBDD siempre vacía. Modificamos la carga inicial de la Base de datos para que reconozca la nueva migración.

Modifica la función `run_initial_sql_migration_if_needed` en `#file:migration.py` para que incorpore al final la ejecución del script en `#file:002_exam_sessions_and_answers.sql`

Cambiamos el código que generó, la carga de las tablas se hacía siempre que no existiese alguna de las tablas, ahora se ejecuta siempre que se cree la tabla de "questions". Base de datos preparada e inicializada si no existe, pasemos ahora a la gestión de la sesión.

Vamos a crear ya el primer punto de la aplicación, la sesión para mantener las 35 preguntas.

Actúa como un programador senior. Crea un nuevo servicio llamado `session_service` encargado de crear una sesión de usuario para llevar la gestión del examen actual. Debe hacer uso del modelo en `#file:exam_session.py` y debe de inicializar la sesión al comienzo de la app usando este nuevo servicio, la sesión se mantendrá activa hasta que el examen se termine.

En la imagen de la derecha vemos un resumen de lo que ha hecho, vamos a comprobarlo y lanzar los test y aplicación por nosotros mismos. Vamos a comprobar cómo se vería en la página principal.

Cambios aplicados:

- Añadí `app/services/session_service.py` con:

- `get_active_session()`
- `create_session()`
- `get_or_create_active_session()`
- `initialize_session()`
- `finish_session()`

- Exporté `SessionService` en `app/services/__init__.py`.

- Modifiqué `app/__init__.py` para:

- inicializar la sesión al arrancar la app,
- guardar el id en `app.config["CURRENT_EXAM_SESSION_ID"]`,
- corregir la URI SQLite en Windows.

- Actualicé `app/blueprints/exam/routes.py` para cerrar la sesión activa al entrar en `/exam/result`.

- Ajusté `app/models/exam_session.py` para usar timestamps UTC con zona horaria.

- Añadí `tests/test_session_service.py` para cubrir:

- creación automática de sesión,
- reutilización de sesión activa,
- cierre correcto de la sesión.

Añade a la template `start.html` la información de la sesión actual

Aparece la información, y parece todo correcto, pero un análisis más detenido del código, nos muestra que **todos** comparten la misma sesión, Si es lo que los requisitos funcionales marcan, perfecto, en otro caso habría que refactorizar el código y crear sesiones diferenciadas por usuarios, mediante una cookie de sesión, un token JWT o una validación obligatoria de usuario.

Sigamos con las preguntas al azar. Seleccionaremos las 35 preguntas del examen para esta sesión.

Modifica la lógica de la siguiente manera:

-- en la ruta raíz, en la función `start_exam` no se seleccionarán 35 preguntas a través del `QuestionBankService.get_questions_for_exam`, se deben crear en `SessionService.get_active_session()`

-- las preguntas se añadirán a la tabla `answer` a través del modelo correspondiente, se inicializarán todas con los valores por defecto usando el id de la sesión recién creada

-- la plantilla `#start.html` seguirá mostrando la misma información.

-- investiga la manera más eficiente

Un poco más de prompts.

Hemos incidido múltiples veces en que una parte muy crítica del proceso de creación con la IA es cómo implementamos los prompts. En general las preguntas breves son ideales para obtener consejos concretos y específicos, para problemas del tipo “¿Cómo puedo consultar esta tabla?” o “¿Cómo puedo solucionar este mensaje de error?”. Las preguntas detalladas son ideales para obtener resultados más abstractos: “¿Cómo puedo diseñar esta interfaz de usuario?” o “¿Cuál es un buen enfoque para organizar los datos?”. Cuando los prompts se vuelven más largos, es mejor darles una estructura que sea fácil de reconocer por el modelo. Una propuesta sería:

Rol: las indicaciones basadas en el rol son algo que ya hemos utilizado anteriormente. Queremos dejar claro que el modelo actúe de una forma específica. Últimamente, parece que establecer un papel no es tan importante como solía serlo.

Objetivo: Establecer claramente nuestro objetivo. Cuanto más precisos y concretos seamos, mejores serán los resultados. Debemos describir exactamente lo que necesitamos.

Detalles: Podemos añadir algunos detalles que serán útiles. Son pequeños, pero significativos para obtener el resultado correcto.

Datos: Describir los datos con los que estamos trabajando y lo que significan. Esto ayuda a generar el código.

Requisitos de funcionalidad: ¿qué queremos que haga? Esta es una buena forma de añadir detalles a nuestro objetivo.

Generación de código: Indicar detalladamente lo que queremos generar.

Navegando por las 35 preguntas

Rol: Actúa como un desarrollador frontend con 5 años de experiencia

Objetivo: Modifica las plantillas y el flujo del programa mostrar una pregunta por pantalla y un progreso del tipo : Pregunta 1 de 35

Detalles: No tengas en cuenta consideraciones de diseño css ni html avanzado

Generación de código: Se modificarán las plantillas html y el código python necesario

Pregunta 2 de 35

What is the limit for transmitter power on the 1.8 MHz band?

- ☐ A. 200 watts PEP output
- ☐ B. 1000 watts PEP output
- ☐ C. 1200 watts PEP output
- ☐ D. 1500 watts PEP output

[Anterior](#) [Siguiente](#) [Revisar](#)

El siguiente lo vamos a dejar intencionadamente un poco más impreciso para ver qué es lo que genera.

Modifica la funcionalidad, para que se guarde en la base de datos los resultados de las preguntas a través del modelo answer al cambiar de pregunta. Si alguna no tiene respuesta se deberá dejar sin modificar.

Modificar un poco la Interfaz para que sea más funcional:

Añade en la plantilla de question un botón para saltar a la primera y otro a la última

La revisión será automática al finalizar.

Elimina el botón revisar de #file:question.html y añade un botón de finalizar examen última pregunta. Este botón estará activo solo cuando todas estén contestadas, mientras estará inactivo

Nuestra experiencia nos dicta que hay un error de lógica, las pruebas funcionan, el movimiento funciona, pero qué pasa al llegar a la última pregunta, no se grabará a menos que retrocedamos, si saltamos al principio tampoco. Vamos a depurar y ver cómo solucionarlo, ahora en modo Pregunta, NO AGENTE.

Examina el código y propón soluciones al problema: La última pregunta no se graba a menos que retroceda de pregunta.

Nos indica el fallo:

El problema: Cuando el usuario presiona "Finalizar examen" en la última pregunta con una respuesta seleccionada:

action = "finish" (del form name="action" value="finish")

"finish" NOT IN {"prev", "next"} → condición en línea 37 es False

La respuesta NO se guarda

Se redirige directamente a /exam/result

Pero no va a solucionar el problema, ya que el botón **Finalizar** no estará nunca activo, ya que solo se habilita cuando estén todas las respuestas grabadas. Le proponemos al **agente** que haga lo siguiente:

Cambia el modo de habilitar el botón finalizar de la plantilla question.html para que esté activo si hay 34 preguntas contestadas y se está en la última pregunta. Además, cambia la lógica para que la última pregunta se grabe al pulsar en el botón finalizar.

No se nos puede olvidar documentar un poco, vamos a generar un gráfico de movimiento.

Haz un diagrama mermaid de:

* Página de inicio (comenzar el examen de práctica):

-Haga clic en «Iniciar examen»

* Aparece una página con la pregunta 1

- Opciones: enviar la respuesta o finalizar el examen

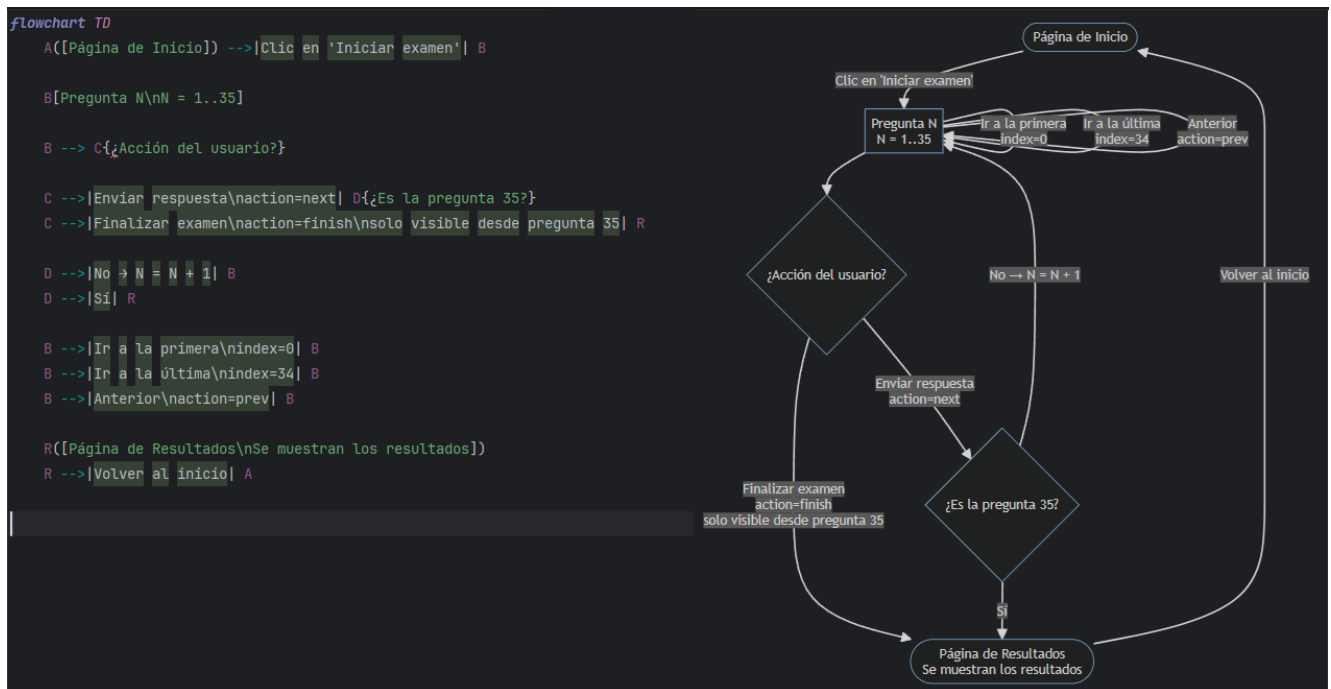
- Si el usuario elige enviar la respuesta, el examen continúa desde la pregunta 1 hasta la 35

- El usuario puede salir del examen en cualquier momento y se le redirigirá a la página de resultados

- Tras la pregunta 35 (examen finalizado)

- El usuario es redirigido a la página de resultados

* Página de resultados: Se muestra la página de resultados con los resultados. Y un enlace para volver a la página de inicio



Casi 600 créditos IA de 1500, menos mal que estamos terminando, está claro que para un programador se quedan muy cortos.

Recordar que el uso más productivo de las herramientas de programación basadas en IA consiste en encontrar el equilibrio entre la automatización y la propia experiencia:

- Utilizar la IA para agilizar las tareas de programación rutinarias y generar soluciones iniciales.
- Aplicar los conocimientos del ámbito para evaluar si las sugerencias se ajustan a los objetivos arquitectónicos.
- Estar preparado para tomar el control manualmente cuando la IA no comprenda del todo el contexto.
- Tratar las herramientas de IA como asistentes colaborativos en lugar de desarrolladores autónomos.
- Reconocer que la IA puede ayudar a encontrar soluciones, pero que se sigue siendo responsable de comprenderlas. No se puede responsabilizar a la IA de los fallos que cometa.

También hay que tener en cuenta que las distintas herramientas de IA destacan en diversas fases del desarrollo:

- Conceptualización inicial: utilizar modelos de lenguaje (LLM) basados en chat para explorar posibilidades de maquetación y patrones de diseño.
- Visualización del flujo: solicitar diagramas en formato Mermaid u otros formatos para trazar los recorridos de los usuarios.
- Creación de esquemas funcionales: pedir esquemas funcionales basados en texto antes de comprometer diseños concretos.
- Generación de HTML/CSS: una vez definido el diseño, solicitar el código de implementación.
- Resolución de problemas: si la implementación presenta problemas, compartir capturas de pantalla o fragmentos de código para obtener sugerencias.
- Documentación: generar descripciones claras de los componentes del interfaz de usuario para los miembros del equipo o los usuarios.
- Tener en cuenta los puntos fuertes de las herramientas: algunas plataformas destacan en la generación de código, mientras que otras ofrecen una mejor orientación conceptual.

Fase 4. Cierre y evaluación.

Cierre final, resultados y revisión

Fase 4 - Cierre y evaluacion (RF-06, RF-07, RF-08)

- Pantalla de confirmacion final con conteo de no respondidas.
 - Junior: 4-8 h
 - Senior: 2-4 h
- Bloquear cambios tras confirmar envio.
 - Junior: 3-6 h
 - Senior: 2-4 h
- Calcular aciertos, errores y porcentaje.
 - Junior: 4-8 h
 - Senior: 2-4 h
- Mostrar resultados (resumen + detalle opcional).
 - Junior: 4-8 h
 - Senior: 2-4 h

La Fase 4, conlleva el cierre del examen, modificaremos la vista de resultados para que muestre el título de cada pregunta, la respuesta seleccionada y un botón para calificar y finalizar el examen.

Modifica la lógica para que en la plantilla `#file:confirm_finish.html` se muestre la descripción de cada pregunta con el texto de la opción seleccionada y la letra correspondiente para cada una,

César San Juan Pastor

además se añadirá un botón de enviar que mande al endpoint `/exam/result` pidiendo confirmación mediante un alert en javascript

Hemos visto que mezcla el código css y decidimos refactorizarlo un poco.

Modifica la plantilla `#file:confirm_finish.html` extrayendo el código css al fichero `exam.css`

Finalizar Examen

Resumen de Respuestas

Preguntas respondidas: **35** de 35

Pregunta 1

Respondida

What does an FT8 signal report of +3 mean?

Tu respuesta: **D** - The signal is 3 dB over S9

Nos ha dejado un aspecto agradable que no tienen las preguntas vamos añadirlo.

Modifica la plantilla `#file:question.html` para que use los mismos estilos que la plantilla `#file:confirm_finish.html`

Y ya de paso la pantalla inicial también.

Da a la plantilla `#file:start.html` un aspecto más profesional, similar al usado en `#file:question.html`

El segundo punto ya se creó de forma automática, en el momento que se cierre el examen será imposible volver a modificar las preguntas. Pasemos a los dos últimos pasos.

Modifica la lógica para que la plantilla `#file:result.html` muestre un recuento de aciertos y fallos, así como que muestre una calificación de apto si supera el 60% de las preguntas. Sigue el estilo visual de las plantillas anteriores. Usa el endpoint `/exam/result`

Ha añadido un botón de revisar el examen que no debe aparecer, lo eliminamos junto con el endpoint `review` y la plantilla. Tras terminar un examen y volver al principio se genera el siguiente error

```
sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) UNIQUE constraint failed: answers.exam_session_id, answers.question_id
[SQL: INSERT INTO answers (exam_session_id, question_id) VALUES (?, ?)]
[parameters: [(3, 'G3B02'), (3, 'G2C01'), (3, 'G8B13'), (3, 'G4E03'), (3, 'G9C09'), (3, 'G1E06'), (3, 'G2B09'), (3, 'G0A07') ... displaying 10 of 35 total bound parameter sets ... (3, 'G5C02'), (3, 'G6A06')]]]
(Background on this error at: https://sqlalche.me/e/20/gkpg)
```

The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error.

To switch between the interactive traceback and the plaintext one, you can click on the "Traceback" headline. From the text traceback you can also create a paste of it. For code execution mouse-over the frame yo

You can execute arbitrary Python code in the stack frames and there are some extra helpers available for introspection:

- `dump()` shows all variables in the frame
- `dump(obj)` dumps all that's known about the object

Vamos a intentar ver porqué.

En la plantilla #file:result.html al pulsar en el botón Volver al inicio se genera el siguiente error

```
sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) UNIQUE constraint failed:
answers.exam_session_id, answers.question_id
```

```
[SQL: INSERT INTO answers (exam_session_id, question_id) VALUES (?, ?)]
```

```
[parameters: [(3, 'G3B02'), (3, 'G2C01'), (3, 'G8B13'), (3, 'G4E03'), (3, 'G9C09'), (3, 'G1E06'), (3,
'G2B09'), (3, 'G0A07') ... displaying 10 of 35 total bound parameter sets ... (3, 'G5C02'), (3,
'G6A06')]]]
```

Busca posibles causas

Ten en cuenta que la sesión ya está finalizada, con una fecha de finalización y el campo status a finished

Propone un par de cambios y parece que funciona. Ahora tenemos que probar nosotros de forma exhaustiva la aplicación.

Para finalizar este punto, ya que la fase cinco se deja para el siguiente tema, veamos un resumen de tiempos:

Fase	Junior (2 años)	Senior (4 años)	Real (IA)
Fase 1 - Base tecnica	2-3 días	1-2 días	10 minutos
Fase 2 - Banco de preguntas	2-4 días	1-2 días	3 horas
Fase 3 - Flujo de examen	5-8 días	3-5 días	1 hora
Fase 4 - Cierre y evaluacion	2-4 días	1-2 días	1 hora
Fase 5 - Calidad	3-5 días	2-3 días	--Falta--
Total estimado	14-24 días	8-14 días	hasta ahora 5 horas aprox

Hay que tener en cuenta que las columnas Junio y Senior también las estimó la IA, en mi opinión este proyecto no debería llevar más de **4-5 días** para un programador Junior, pero que se podría acelerar hasta 1-2 días con el uso de la IA.

Apreciación personal: Tras el desarrollo final, a falta de la Fase 5 de pruebas, encontramos que la IA es muy útil, pero guiándola y sobre todo en las etapas iniciales. En este ejemplo ha creado una estructura muy compleja típica de aplicaciones profesionales, pero poco asumible para este ejemplo. Se debería limitar mucho la acción en las fases iniciales con procesos de verificación muy completos. Para el resto de código, si los prompts son adecuados y muy explícitos, indicando una única tarea en general responde muy bien, pero siempre con nuestra supervisión.

Ejercicio

Añadir soporte Github

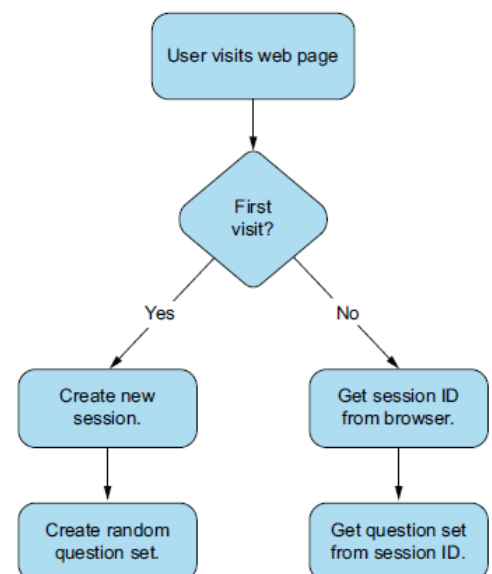
Se pretende subir la aplicación a un repositorio GitHub, utilizando el asistente, con un repositorio ya creado, inicializar todo el código creado que tenemos hasta ahora en el repositorio.

A partir de ahora, usar el agente para que cree los comit y los pull cada vez que terminemos algún desarrollo.

Modificar para que se cree un nuevo examen si no hay uno ya existente para cada usuario

Si recordamos el párrafo de la fase 3: *Vemos la información, y parece todo correcto, pero un análisis más detenido del código, nos muestra que **todos** comparten la misma sesión, Si es lo que los requisitos funcionales marcan, perfecto, en otro caso habría que refactorizar el código y crear sesiones por usuarios, mediante una cookie de sesión, un token JWT o una validación obligatoria de usuario.*

Modificar la aplicación para que desde el cliente se envíe una cookie con el número de sesión, no hace falta usar tecnologías avanzadas tales como JWT o tabla de usuarios y sesiones, de momento no nos importa la seguridad. Si el navegador entrega la cookie con un número de sesión, la aplicación entenderá que se quiere continuar con dicho examen, mostrándolo por la primera pregunta, si está terminado se informará de un error y se mostrarán los resultados, borrando la cookie del navegador.



Modificar para mejora el UI

Cambiar el diseño de la aplicación para que use algún Framework CSS como Tailwind o Bootstrap. La aplicación debe ser responsive, se debe hacer un proceso de ingeniería primero:

1. Analizar con la IA los RF y posibles soluciones gráficas
2. Diseñar las diferentes plantillas en wireframe con la IA
3. Modificar a partir de plantillas generadas las templates actuales sin cambiar el código ejecutable, solo el aspecto.

Tema 6 – Otras necesidades

Depuración con IA

La depuración asistida por IA consiste en utilizar herramientas como GitHub Copilot Chat dentro del entorno de desarrollo (como VS Code) para que el modelo analice y resuelva errores. En lugar de buscar soluciones a ciegas, el LLM utiliza el contexto del proyecto para ofrecer explicaciones y soluciones precisas.

Para usarlo eficazmente, se debe alimentar al modelo con el contexto adecuado. Cuando se encuentra un error, se utiliza el chat integrado y hay que añadir enlaces específicos para señalar el archivo, fragmento de código exacto o el mensaje de error. También se pueden emplear comandos directos como `/fix` o `/explain` para obtener una propuesta de corrección inmediata y entender la raíz del problema.

La depuración alcanza su máximo potencial al integrarse con el depurador local. Si pausamos la ejecución del programa mediante breakpoints, podemos compartir el estado de las variables y el call stack con la IA. Esto le permite al modelo analizar el comportamiento en tiempo real, identificar qué variable tiene un valor incorrecto y sugerir cómo ajustar la lógica paso a paso.

Desarrollo de test con la IA

Las pruebas de software son fundamentales para garantizar la fiabilidad y la seguridad, pero a menudo se realizan con prisas debido a las presiones del desarrollo. Es posible crear automáticamente conjuntos de pruebas detallados, detectar casos extremos y generar el código repetitivo que contienen muchos de los test que se desarrollan. La IA generativa no solo agiliza el proceso, sino que también mejora la calidad de las pruebas. Pero como todo el discurso que llevamos, sin supervisión no generará valor, ya que puede contener fallos, crear desviaciones, no testear todos los casos necesarios e incurrir en problemas legales, sobre todo de copyright.

Al igual que los prompt anteriores, se deberían seguir las mismas reglas vitas hasta ahora, así un prompt del tipo:

Ayúdame a escribir pruebas con pytest para el archivo `/models/questions.py`.

Es muy generalista y puede generar errores, si bien como todo modelo nos dará soluciones es muy probable que incurra en múltiples fallos. Probablemente una mejor aproximación sería:

Crea pruebas unitarias con pytest para la clase «questions», ubicada en `/app/models/questions.py`. La aplicación utiliza una base de datos SQLite, y me gustaría crear una base de datos en memoria que se corresponda con la base de datos de producción con fines de prueba.

Deberíamos seguir las siguientes reglas

- Indicar las rutas de los archivos para ayudar a la IA a localizar el código.
- Mencionar los requisitos de la base de datos (en memoria o simulada).
- Describir los casos extremos que deseamos que se prueben.
- Solicitar estilos de afirmación específicos.
- Incluir información sobre las dependencias.

- Mencionar los fixtures, indicando si deben reutilizarse o crearse.
- Empezar con algo general y luego refinar con indicaciones adicionales si es necesario.
- Por ejemplo, en lugar de “crea pruebas para mi función”, probar con “Crea pruebas Pytest para el método `get_question_set` en `app/models/questions.py` utilizando el fixture de base de datos en memoria. Incluye pruebas tanto para resultados vacíos como para el funcionamiento normal”.

A la hora de generar pruebas unitarias, la creación para conexiones a servidores externos suele ser las que nos presentan mayor número de problemas, hay que tener en cuenta que o bien gestionamos un servidor de pruebas o realizamos mock y gestionamos en memoria. Desde este punto de vista, la segunda aproximación tiene las siguientes ventajas:

- **Velocidad:** las operaciones en memoria son considerablemente más rápidas que las pruebas basadas en disco.
- **Aislamiento:** las pruebas se ejecutan en un entorno aislado sin afectar a los datos de producción.
- **Consistencia:** cada prueba comienza con un estado de la base de datos limpio y predecible.
- **Sin necesidad de limpieza:** la base de datos desaparece una vez finalizadas las pruebas.
- **Sin configuración:** no se necesitan credenciales ni conexiones de base de datos de prueba independientes.
- **Simplicidad:** el código de prueba se parece más al código de producción, lo que mejora la mantenibilidad.

Pero, por el contrario, los datos serán generalmente sintéticos y tendrán poco que ver con lo que nos encontremos posteriormente en producción.

Fase 5.

Comprobemos primero errores de seguridad en la aplicación.

Actúa como un auditor de seguridad con dos años de experiencia y realiza un análisis de riesgos, centrados en: transmisión de datos en los endpoints, sql injections, patrones inseguros y código con posibles fallos. Genera un informe en markdown.

El informe es extenso, ver: **informe_seguridad.md**. Como relevantes aparecen varios errores que deberíamos tener ya detectados, el desarrollo no estaba orientado a la seguridad. Se deberían reparar si no todos, la mayoría, de los graves y medios todos.

Vamos a crear las pruebas unitarias que faltan:

Realiza los test unitarios de los servicios que no los tengan. Ten en cuenta los casos límite, usa datos en memoria si fuera necesario.

A continuación, testeamos todas las rutas

Crea los test necesarios para todos los endpoints que se usan de `#file:routes.py`, utilizando mock con datos en memoria, no uses la base de datos real de la app

Para terminar con los test comprobaremos los casos límite

Comprueba los test existentes para ver si los casos extremos: menos de 35 preguntas, sesión inválida, doble envío se tienen en cuenta

Con el prompt anterior detecta que no están cubiertos y nos sugiere crearlos, a lo que accedemos.

crea los test faltantes

A modo de resumen, y teniendo en consideración lo hablado antes, el recuento final de tiempos sería el siguiente:

Fase	Junior (2 años)	Senior (4 años)	Real (IA)
Fase 1 - Base tecnica	2-3 dias	1-2 dias	10 minutos
Fase 2 - Banco de preguntas	2-4 dias	1-2 dias	3 horas
Fase 3 - Flujo de examen	5-8 dias	3-5 dias	1 hora
Fase 4 - Cierre y evaluacion	2-4 dias	1-2 dias	1 hora
Fase 5 - Calidad	3-5 dias	2-3 dias	1 hora
Total estimado	14-24 dias	8-14 dias	6 horas aprox

Con todos los test generados, deberíamos ahora revisar el código creado y comprobar que al menos la cobertura es adecuada.

Ejecuta los test con análisis de cobertura, e indica mejoras en aquellos casos en la que no llegue al 80%.

Después de los resultados, decidimos ampliar los test para llegar al menos a la cobertura deseada.

Segun el informe anterior, haz:

- añade tests a question_bank_service.py
- añade tests a exam_repository.py
- NO incluyas test para los ficheros que tengan en su nombre `__DELETED__`

Creamos un informe final de cobertura para añadir a la documentación.

Haz un informe final de cobertura en formato markdown

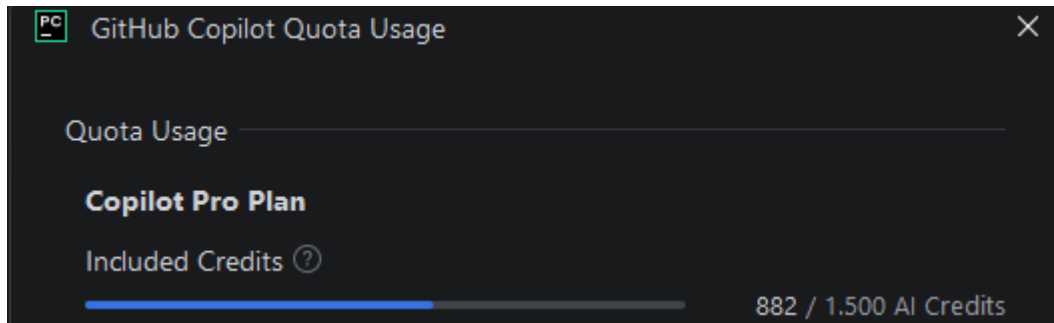
Podemos ver el resultado en el fichero `informe_cobertura.md`.

César San Juan Pastor

Para terminar, sincronizamos el fichero de requerimientos para producción.

Sincroniza `#file:requirements.txt` para que el proyecto se reproduzca.

Para este proyecto, muy simple, y con unas seis horas de trabajo hemos usado más de 850 créditos, lo que vendría a ser unos dos o tres días de trabajo real.



Extrapolando el resultado a proyectos más complejos necesitaríamos unos 10.000 créditos de uso, unas seis veces de la cuota asignada al plan actual.

Tema 7 – Otras consideraciones

Comparativa de los diferentes agentes

Modelo	Fortalezas	Debilidades	Casos de uso más comunes en programación
GPT-5.5	Excelente razonamiento, muy buena comprensión de bases de código grandes, generación consistente, fuerte en depuración y arquitectura	Coste superior frente a modelos pequeños; puede ser excesivo para tareas simples	Desarrollo full-stack, revisión de código, refactorización compleja, diseño de arquitectura, generación de pruebas
GPT-5 Mini	Muy rápido, económico, buena calidad general	Menos capacidad en razonamiento complejo y contextos enormes	Autocompletado, scripts, automatizaciones, soporte en IDE
Claude Opus 4	Excelente comprensión de repositorios extensos, análisis profundo, documentación técnica	Algo más lento; ocasionalmente demasiado conservador en respuestas	Refactorización masiva, análisis de código legacy, documentación, revisiones de PR
Claude Sonnet 4	Muy buena relación calidad-rendimiento, fuerte en programación y razonamiento	Menor capacidad que Opus en tareas extremadamente complejas	Desarrollo diario, pair programming, generación de código y debugging
Gemini 2.5 Pro	Gran ventana de contexto, análisis de múltiples archivos, buena integración con ecosistema Google	Puede ser inconsistente en detalles de implementación fina	Proyectos grandes, análisis de documentación extensa, arquitecturas empresariales
Gemini 2.5 Flash	Muy rápido y económico	Menos precisión en tareas complejas	Chat de soporte para desarrolladores, generación rápida de código simple
DeepSeek R1	Excelente razonamiento por coste, muy fuerte en algoritmos y resolución de problemas	Menor calidad en interacción y mantenimiento de contexto largo	Algoritmos, estructuras de datos, preparación de entrevistas técnicas
DeepSeek V3	Muy competitivo en programación general y bajo coste	Ecosistema e integraciones más limitados	Desarrollo backend, generación de funciones, automatización
Qwen 3	Buen rendimiento open-weight, multilingüe, fuerte en código	Menor consistencia que los modelos líderes	Desarrollo open source, despliegues locales, asistentes internos
Llama 4	Open-weight, personalizable, fácil de ejecutar on-premise	Calidad variable según versión y ajuste	Copilot privado, herramientas empresariales internas, inferencia local
Codestral	Especializado en generación de código, rápido	Menor capacidad de razonamiento general	Autocompletado, generación de funciones, integración IDE
Devstral	Optimizado para tareas de ingeniería de software y agentes	Menos probado a gran escala que GPT o Claude	Agentes de desarrollo, mantenimiento de repositorios, workflows CI/CD

La comparativa a nivel general quedaría:

Categoría	Mejor opción
Programación general	GPT-5.5
Repositorios muy grandes	Claude Opus 4
Relación calidad/precio	Claude Sonnet 4
Algoritmos y competición	DeepSeek R1
Uso local (on-premise)	Llama 4 o Qwen 3
Autocompletado rápido	Codestral
Análisis documental enorme	Gemini 2.5 Pro
Agentes autónomos de desarrollo	GPT-5.5, Claude Opus 4 y Devstral

Según el perfil

Perfil	Modelo recomendado
Desarrollador individual	Claude Sonnet 4 o GPT-5.5
Startup	GPT-5.5 + modelos económicos como DeepSeek V3
Empresa con requisitos de privacidad	Llama 4 o Qwen 3 desplegados localmente
Entrevistas técnicas	DeepSeek R1
Desarrollo full-stack profesional	GPT-5.5 o Claude Opus 4

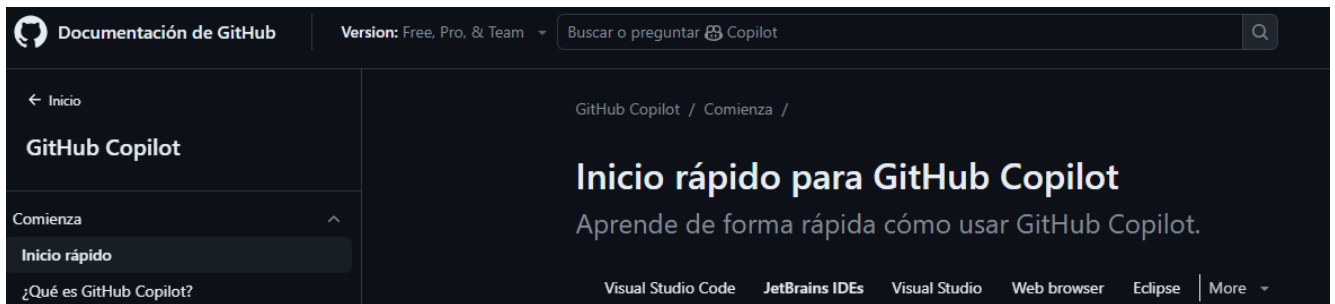
La disponibilidad de estos potentes asistentes de programación basados en IA supone un cambio fundamental en las prácticas de desarrollo de software. En lugar de considerarlos opciones que compiten entre sí, los desarrolladores reconocen que cada familia de modelos aporta fortalezas únicas a diferentes aspectos del proceso de desarrollo. Gemini, de Google, destaca cuando el contexto visual y la comprensión multimodal son cruciales, especialmente en el desarrollo de UI/UX y al trabajar con especificaciones de diseño. Claude, de Anthropic, destaca en situaciones que requieren un razonamiento profundo, refactorizaciones complejas y enfoques transparentes para la resolución de problemas. La familia de modelos de OpenAI ofrece una versatilidad inigualable y amplios conocimientos, lo que la hace ideal para el aprendizaje, la depuración y los retos interdisciplinarios.

Muchos equipos de desarrollo emplean ahora un enfoque de cartera, aprovechando diferentes modelos para distintas tareas dentro del mismo proyecto. Un flujo de trabajo típico podría implicar el uso de Gemini para traducir maquetas de diseño en implementaciones iniciales, de Claude para decisiones arquitectónicas complejas y revisiones de código, y de ChatGPT para la resolución general de problemas y la documentación. Este enfoque multimodelo maximiza la productividad al adaptar los puntos fuertes de cada herramienta a retos de desarrollo específicos.

Github Copilot

Según diversas páginas de Internet, y tras leer varios artículos, he llegado a la conclusión que el estándar de facto para la programación asistida con IA bajo IDE es GitHub Copilot. Este agente no es tan avanzado como los integrados en IDEs específicos tipo Cursor que es capaz de manejar varios agentes simultáneos y delegar en subagentes, pero establece una buena relación entre la ayuda y la generación Code Vibe.

La url anterior nos da acceso a la última documentación a día de hoy, en este momento tan cambiante no podemos asegurar que siga siendo la misma cuando se lea este párrafo.



Es un buen ejercicio leer la documentación para tener conocimiento de las últimas novedades y usos del mismo.

Una visión al futuro que nos viene

Ojalá pudiéremos intuir, avanzar o saber qué nos depara el futuro, y menos en estos momentos tan incipientes. Se ha llegado a decir que en un par de meses ya no se escribirá más código, en vez de adentrarnos en un ejercicio de adivinación, vamos a centrarnos en aspectos concretos del desarrollo y como podrían influir en los próximos días, meses, años, quién sabe:

- Los agentes de IA pasarán a formar parte habitual del equipo de desarrollo.
- Los puntos de control inteligentes permitirán a las IA pedir ayuda a los humanos.
- La experiencia de usuario de los agentes mejorará.
- Las mejoras en los modelos reducirán la brecha del 30%. Posiblemente siempre habrá cierto margen para el criterio humano. Quizá ese margen pase a ser del 5% al 10%, en lugar del 30%.
- Los agentes y las herramientas se volverán más especializados.
- Los desarrolladores experimentarán un cambio cultural y de competencias. En esencia, el “30% humano” se centrará en el pensamiento crítico de alto nivel y en los aspectos de control de calidad del desarrollo de software. Veremos cambios en la forma en que los desarrolladores junior se ponen al día. Quizá empiecen gestionando un agente de IA en tareas sencillas antes de escribir ellos mismos una gran cantidad de código, lo que podría ser tanto positivo (pueden aportar valor rápidamente) como un reto (deberán aprender los fundamentos y no considerar la IA como un apoyo). Es un momento emocionante para aquellos que estén dispuestos a adaptarse, pero puede resultar incómodo para quienes prefieren la forma tradicional.
- Surgirán nuevas funciones y procesos

Nuestro sector debe integrar estos cambios con cuidado. El elemento con su creatividad, intuición y juicio ético— sigue siendo insustituible. La IA puede potenciar nuestras capacidades, pero también puede amplificar los errores si no se controla.

Y los lenguajes de programación qué: no creemos que los lenguajes de programación vayan a desaparecer de la noche a la mañana. En cambio, lo que podría ocurrir es una estructura por capas: el lenguaje natural para la coordinación de alto nivel y los lenguajes de programación existentes “entre

bastidores” para un control más detallado. Una de las razones por las que existen los lenguajes de programación es que el lenguaje natural puede resultar ambiguo. Otra posibilidad son los lenguajes híbridos que combinan el lenguaje natural y el código.

Quizás surjan nuevos paradigmas de programación que sean intrínsecamente compatibles con la IA, es decir, que dejen margen para que la IA complete los huecos; por ejemplo, un lenguaje que permita programas parciales con marcadores de posición que una IA pueda resolver («[Optimizar aquí para ganar velocidad]») o con lógica difusa que la IA pueda refinar hasta convertirla en lógica.

Ejercicio

El desarrollo de los capítulos anteriores ha estado muy guiado por la IA, se propone realizar una aplicación diferente pero ahora con menos soporte de la IA, solo para crear código puntual en diferentes funciones, métodos o clases, siendo nosotros los que desarrollemos la mayoría del código, sobre todo la estructura inicial de directorios y ficheros.

Anexo I – Recursos

Bibliografía

1. Coding with IA. Jeremy C. Morgan, ISBN: 9781633437272. Manning
2. Beyond Vibe Coding From Coder to AI-Era Developer, Addy Osmani, ISBN: 979-8-341-63475-6, O'Reilly
3. Supercharged Coding with GenAI, Hila Paz Herszfang, _ISBN:978-1-83664-529-0, Packt